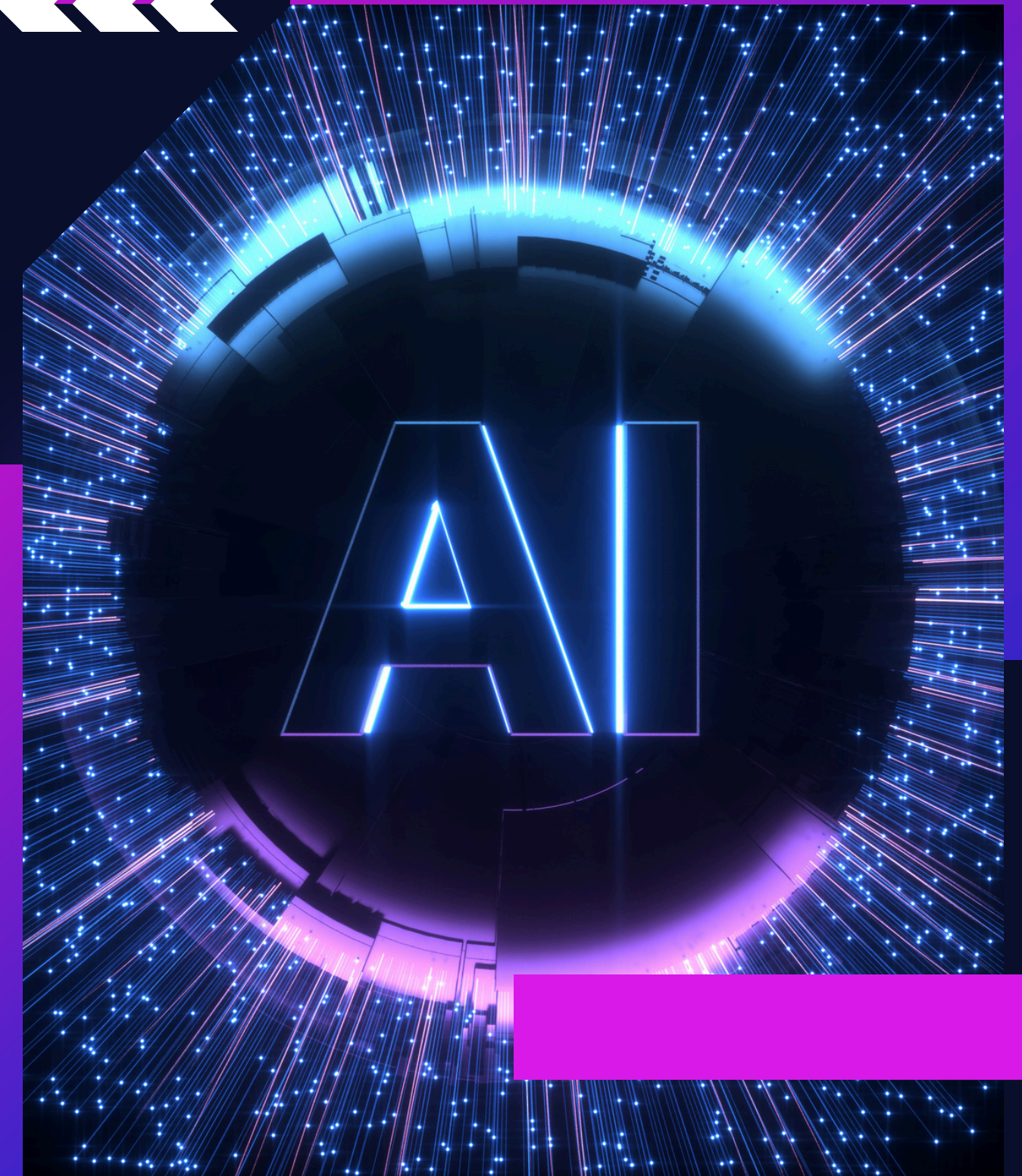


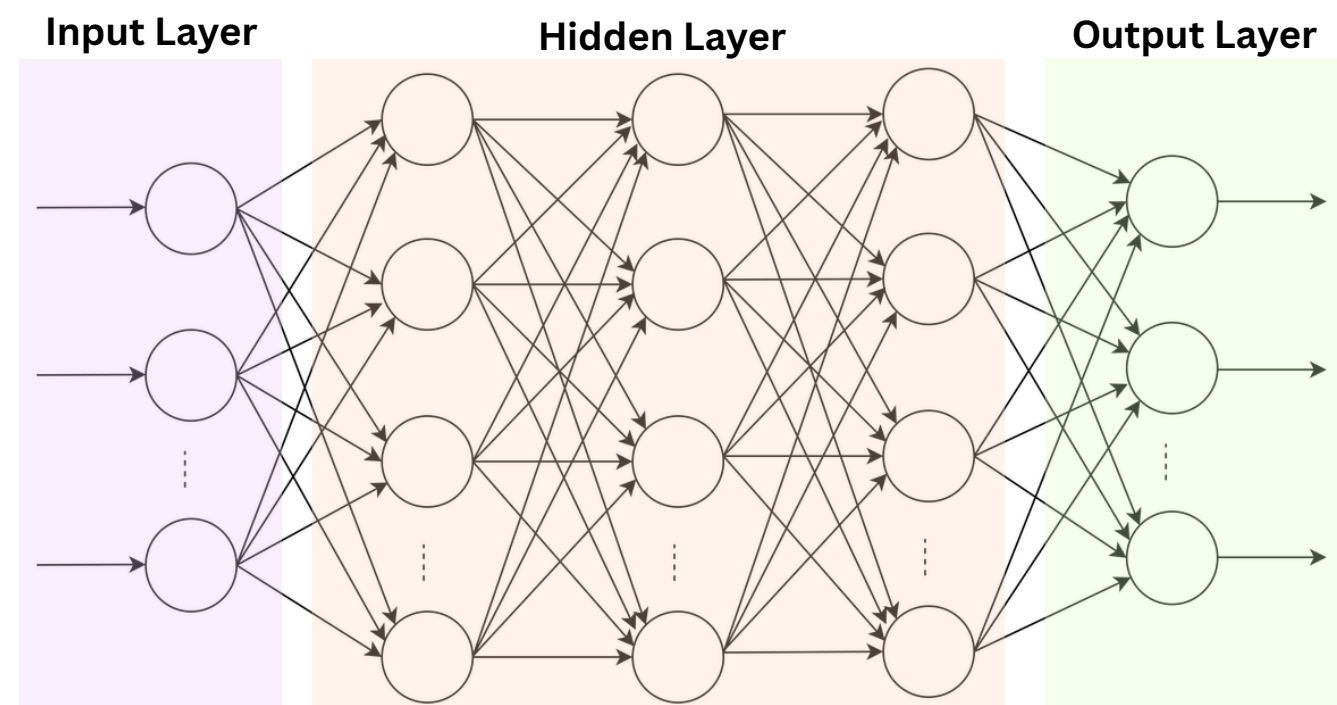
บทที่ 4 : การเรียนรู้เชิงลึก

- 4.1 แนะนำการเรียนรู้เชิงลึก (DL)
- 4.2 โครงข่ายประสาทเทียม (ANN)
- 4.3 มัลติเลเยอร์ เพอร์เซปตรอน (MLP)
- 4.4 คอนโวลูชันนัล นิวรัลเน็ตเวิร์ก (CNN)
- 4.5 รีเคอร์เรนต์ นิวรัลเน็ตเวิร์ก (RNN)
- 4.6 ลองชอร์ตเทอมเมมโมรี (LSTM)
- 4.7 ดีป คิว-เน็ตเวิร์ก (DQN)
- 4.8 เจเนอเรทีฟ พรินทรด์ ทรานส์ฟอร์มเมอร์ (GPT)
- 4.9 เจเนอเรทีฟ แอดเวอร์ซารีล เน็ตเวิร์ก (GAN)
- 4.10 แบบฝึกหัด (Exercise)

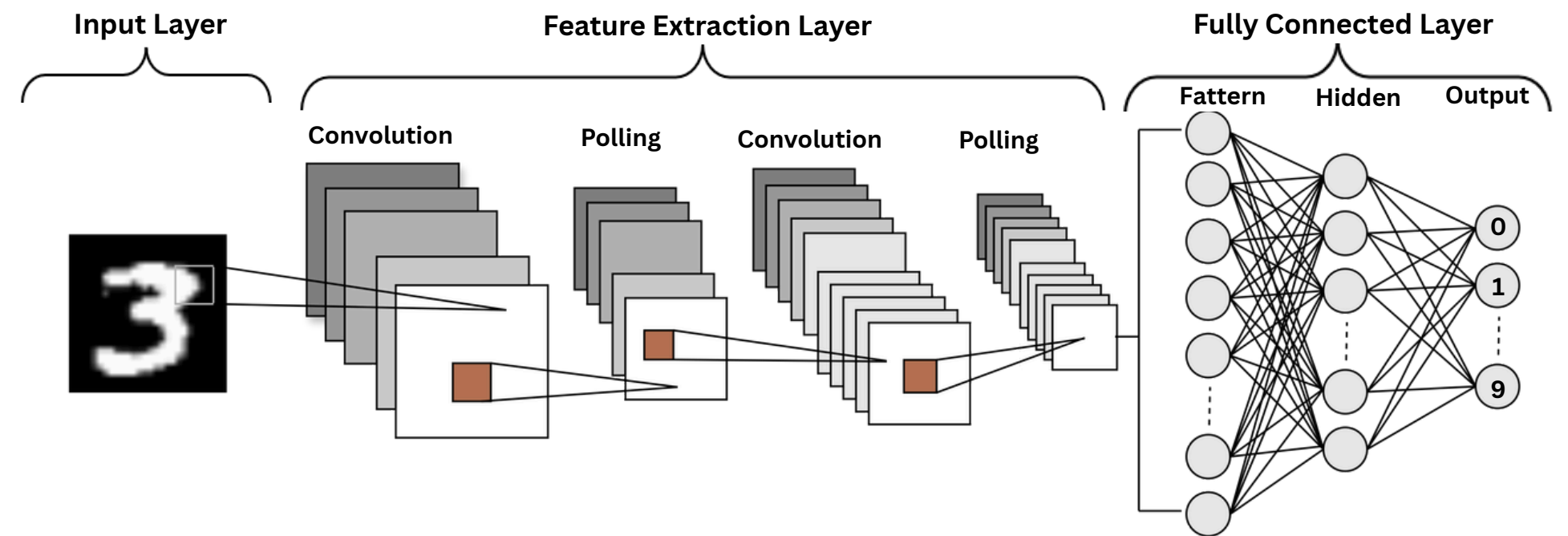


4.1 แนะนำการเรียนรู้เชิงลึก (DL)

การเรียนรู้เชิงลึก (Deep Learning: DL) เป็นเทคนิคย่อยหนึ่งในสาขาการเรียนรู้ของเครื่อง (ML) ที่มีโครงสร้างหลายชั้น (Multilayer) มักใช้โครงข่ายประสาทเทียม (ANN) ในการเรียนรู้จากข้อมูลขนาดใหญ่ การเรียนรู้เชิงลึกได้รับการพิสูจน์แล้วว่ามีประสิทธิภาพในงานที่เกี่ยวข้องกับภาพ (Image) การรับรู้เสียงพูด (Speech Recognition) การประมวลผลภาษาธรรมชาติ (NLP) และเกมส์เชิงยุทธศาสตร์ (Strategic Games)



ภาพที่ 4-1ก : โครงสร้างของ MLP



ภาพที่ 4-1ข : โครงสร้างของ CNN

ภาพที่ 4-1 : โครงสร้างของ MLP และ CNN

ข้อดี

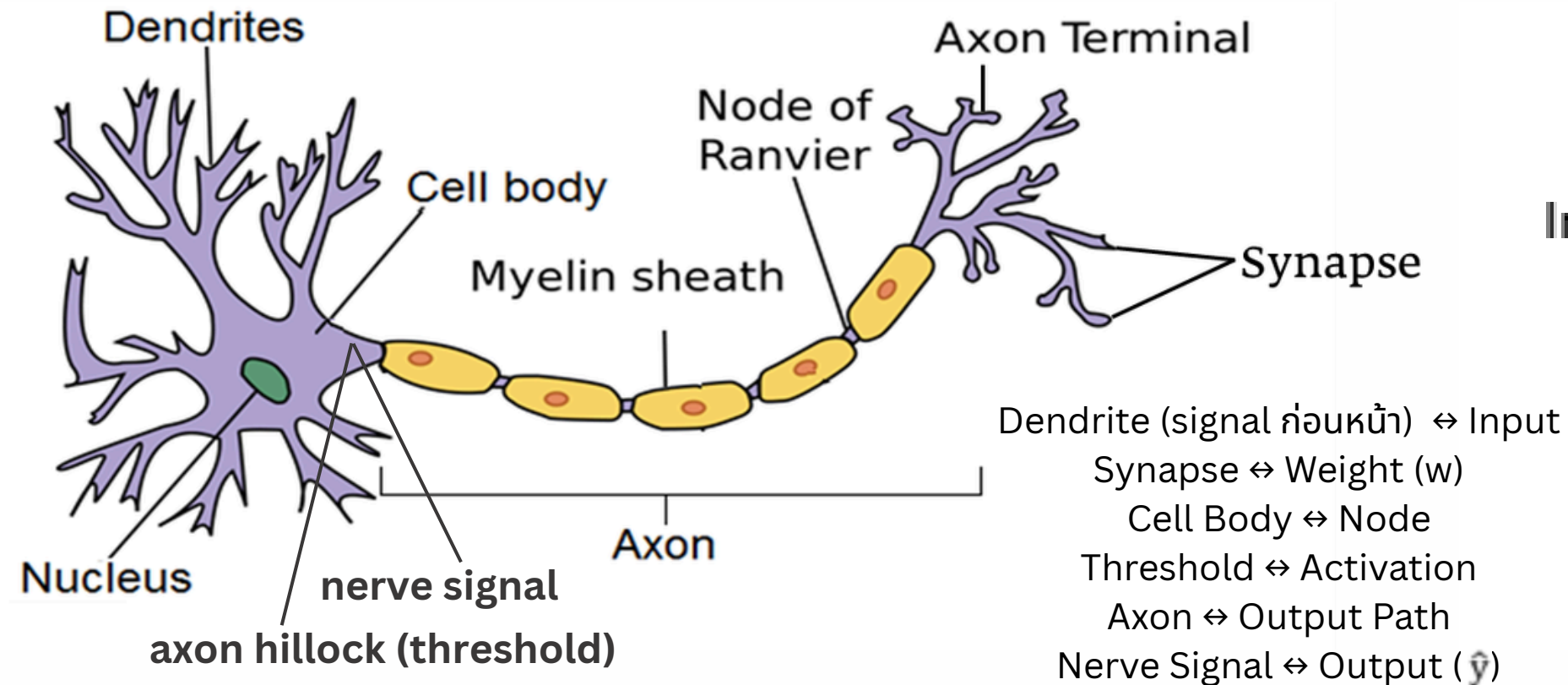
- **มีความถูกต้องสูง (Accuracy)** – เมื่อมีข้อมูลจำนวนมาก, DL มักให้ผลลัพธ์ดีกว่าวิธี ML แบบดั้งเดิม
- **มีความยืดหยุ่น (Flexibility)** – สามารถปรับใช้ได้กับหลายโดเมน เช่น การแพทย์ (วินิจฉัยโรค), การเงิน (ตรวจจับการโกง), ระบบอัตโนมัติ (รถยนต์ไร้คนขับ)
- **สกัดฟีเจอร์อัตโนมัติ (Feature Extraction)** – สามารถสกัดฟีเจอร์ของภาพหรือข้อมูลได้อัตโนมัติ เช่น การสกัดหาเส้นขอบ ตา, คิ้ว, จมูก, และปาก ในภาพใบหน้า เพื่อใช้เป็นองค์ประกอบในการเรียนรู้ใบหน้าคน

ข้อเสีย

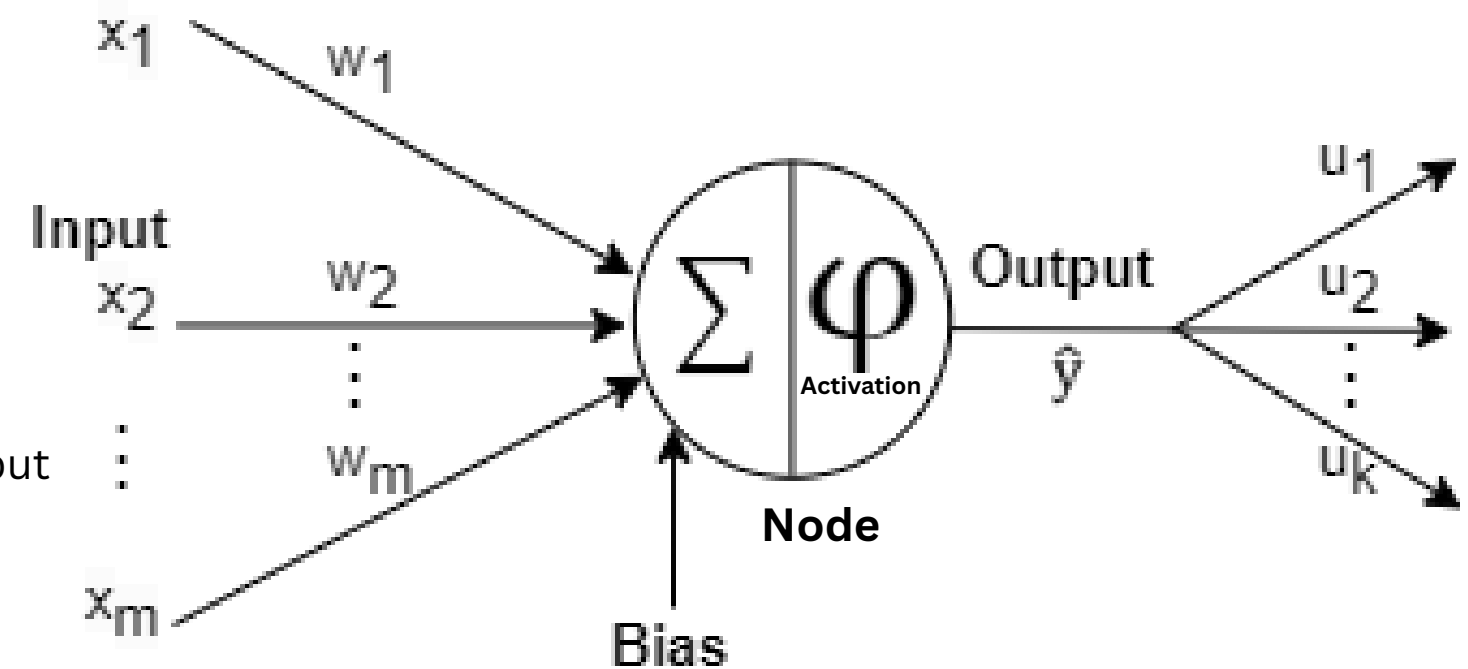
- **ต้องการข้อมูลจำนวนมาก (Big Data)** – DL มักทำงานได้ดีเมื่อมีข้อมูลจำนวนมาก หากข้อมูลมีน้อย DL อาจไม่แม่นยำเนื่องจาก DL มีโครงสร้างที่ซับซ้อน
- **ต้องการทรัพยากรคอมพิวเตอร์สูง (Resource-intensive)** – ต้องใช้ GPU/TPU และหน่วยความจำมาก ทำให้ค่าใช้จ่ายสูงทั้งในด้านฮาร์ดแวร์และพลังงาน รวมไปถึงการใช้เวลาในการประมวลผลที่นาน
- **ขาดความสามารถในการตีความ (Lack of Interpretability)** – DL มีความเป็น black box เนื่องจากมันยากที่จะอธิบายได้ว่าโมเดลตัดสินใจอย่างไร ซึ่งเป็นข้อจำกัดในงานที่ต้องการความน่าเชื่อถือสูง เช่น การวินิจฉัยทางการแพทย์

4.2 โครงข่ายประสาทเทียม (ANN)

โครงข่ายประสาทเทียม (Artificial Neural Network: ANN) คือ โครงข่ายประสาทเทียมที่สามารถเลียนแบบการทำงานของสมองมนุษย์ ใช้ในการประมวลผลข้อมูลและเรียนรู้รูปแบบ (patterns) จากข้อมูลที่ป้อนเข้าไป



ภาพที่ 4-2ก : Biological Neuron



ภาพที่ 4-2ข : Artificial Neuron

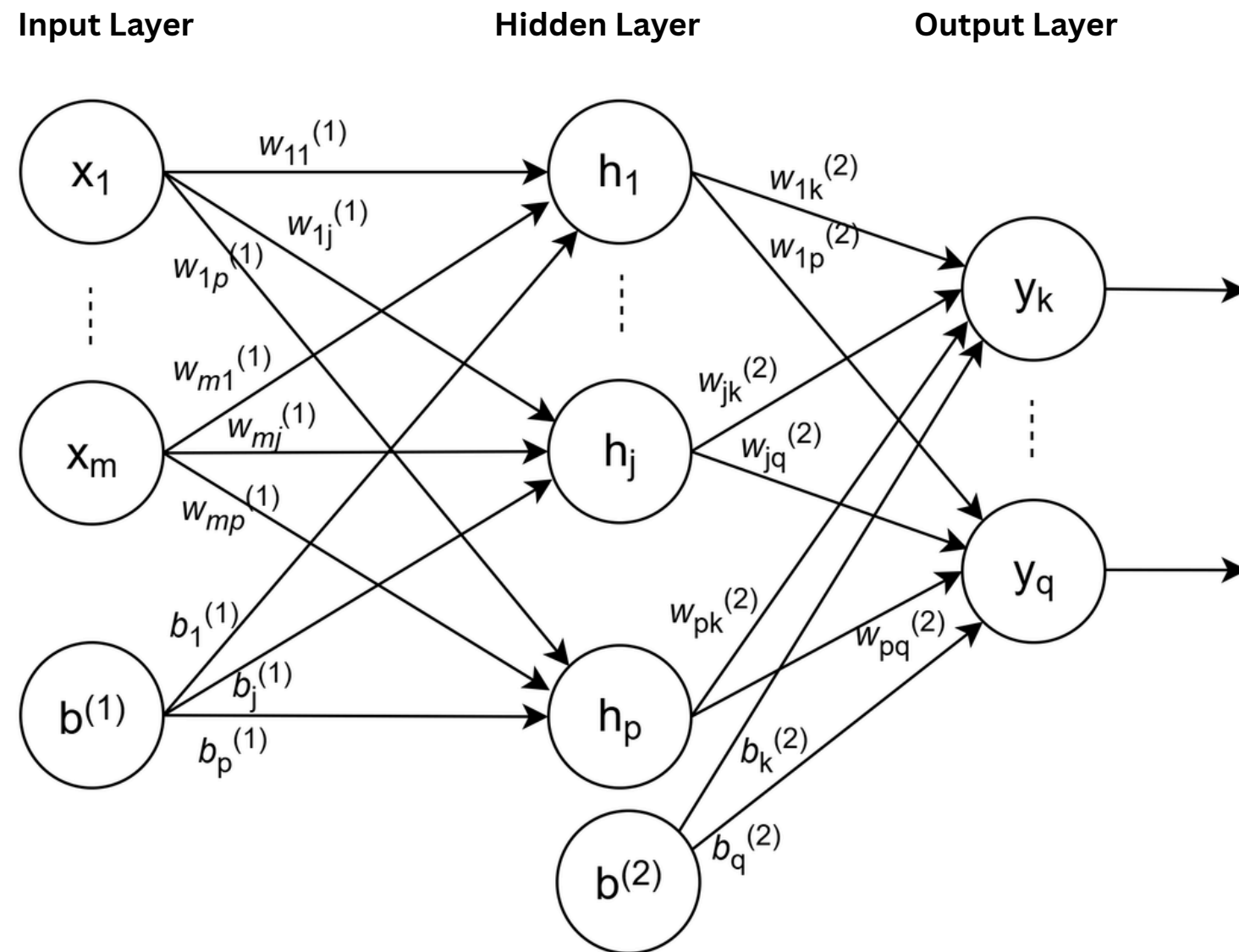
ภาพที่ 4-2 : เปรียบเทียบ Biological Neuron กับ Artificial Neuron

4.3 มัลติเลเยอร์ เพอร์เซ็ปตรอน (MLP)

มัลติเลเยอร์ เพอร์เซ็ปตรอน (Multilayer Perceptron: MLP)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Feed forward neural network
 - รับข้อมูลเข้าในทิศทางจาก Input → Hidden Layer(s) → Output
 - ใน output layer, คำนวณผลลัพธ์ (predicted) แล้วนำไปเทียบกับค่าจริง (actual) เพื่อคำนวณ Error
- เรียนรู้โดยใช้ Backpropagation
 - ย้อนกลับไปปรับ weight ใหม่ (อัปเดตความรู้) โดยใช้ error ที่คำนวณได้
- เป็น Universal Approximator
 - ในปี 1989 George Cybenko ได้พิสูจน์ว่า ANN ที่มีเพียง 1 Hidden Layer และใช้ Sigmoid เป็น Activation Function สามารถประมาณฟังก์ชันต่อเนื่องได้ทุกชนิด

4.3.1 โครงสร้างของ MLP



ภาพที่ 4-3 : โครงสร้างของ MLP

$$\mathbf{h} = \varphi(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$h_j = \varphi(w_{1j}^{(1)}x_1 + \dots + w_{mj}^{(1)}x_m + b_j^{(1)})$$

h_j คือ hidden node ที่ j

φ คือ activation function (เช่น ReLU หรือ Sigmoid)

$w_{1j}^{(1)}$ คือ ค่า weight ที่เชื่อมจาก input node ที่ 1 ไปยัง hidden node ที่ j

x_1 คือ ข้อมูล input node ที่ 1 ซึ่งมีทั้งหมด m โหนด

$b_j^{(1)}$ คือ ค่า bias ของ hidden node ที่ j

$$\hat{\mathbf{y}} = \varphi(\mathbf{w}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

$$\hat{y}_k = \varphi(w_{1k}^{(2)}h_1 + \dots + w_{jk}^{(2)}h_j + w_{pk}^{(2)}h_p + b_k^{(2)})$$

$\hat{y}_k$ คือ output node ตัวที่ k

φ คือ activation function เช่น Sigmoid สำหรับ binary class,

Softmax สำหรับ multi-class, และ linear/ReLU สำหรับ regression

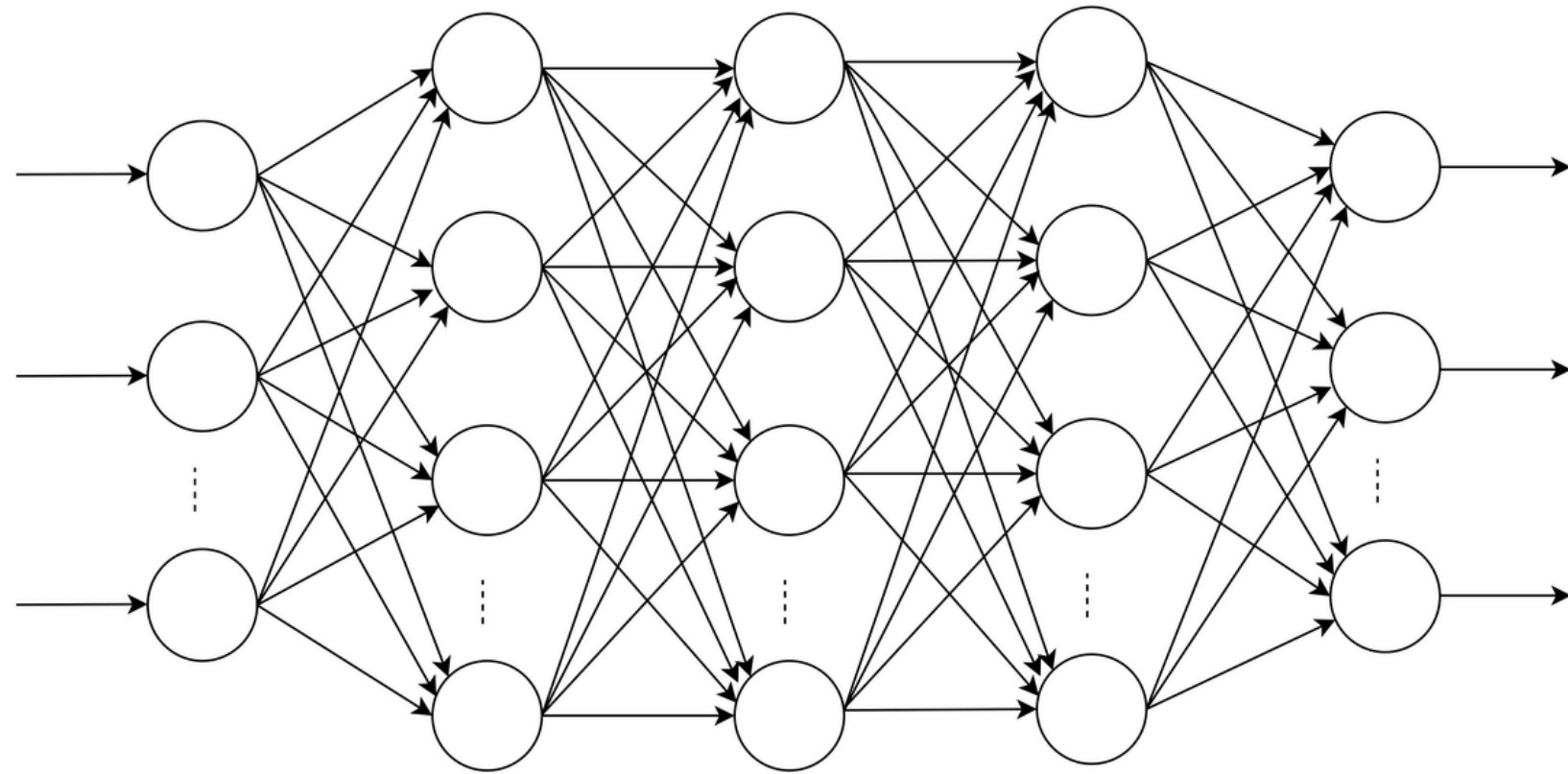
$w_{1k}^{(2)}$ คือ ค่า weight ที่เชื่อมจาก hidden node ที่ 1 ไปยัง output node ที่ k

h_1 คือ hidden node ที่ 1 ซึ่งมีทั้งหมด p โหนด

$b_k^{(2)}$ คือ ค่า bias ของ output node ที่ k

4.3.2 การคำนวณสำหรับ MLP

ขั้นตอนการเทรน (Train) โมเดล



1. forward → 2. loss (error) → 3. backprop → 4. adjust weight

ขั้นที่ 0 : กำหนดสถาปัตยกรรมและค่าเริ่มต้น

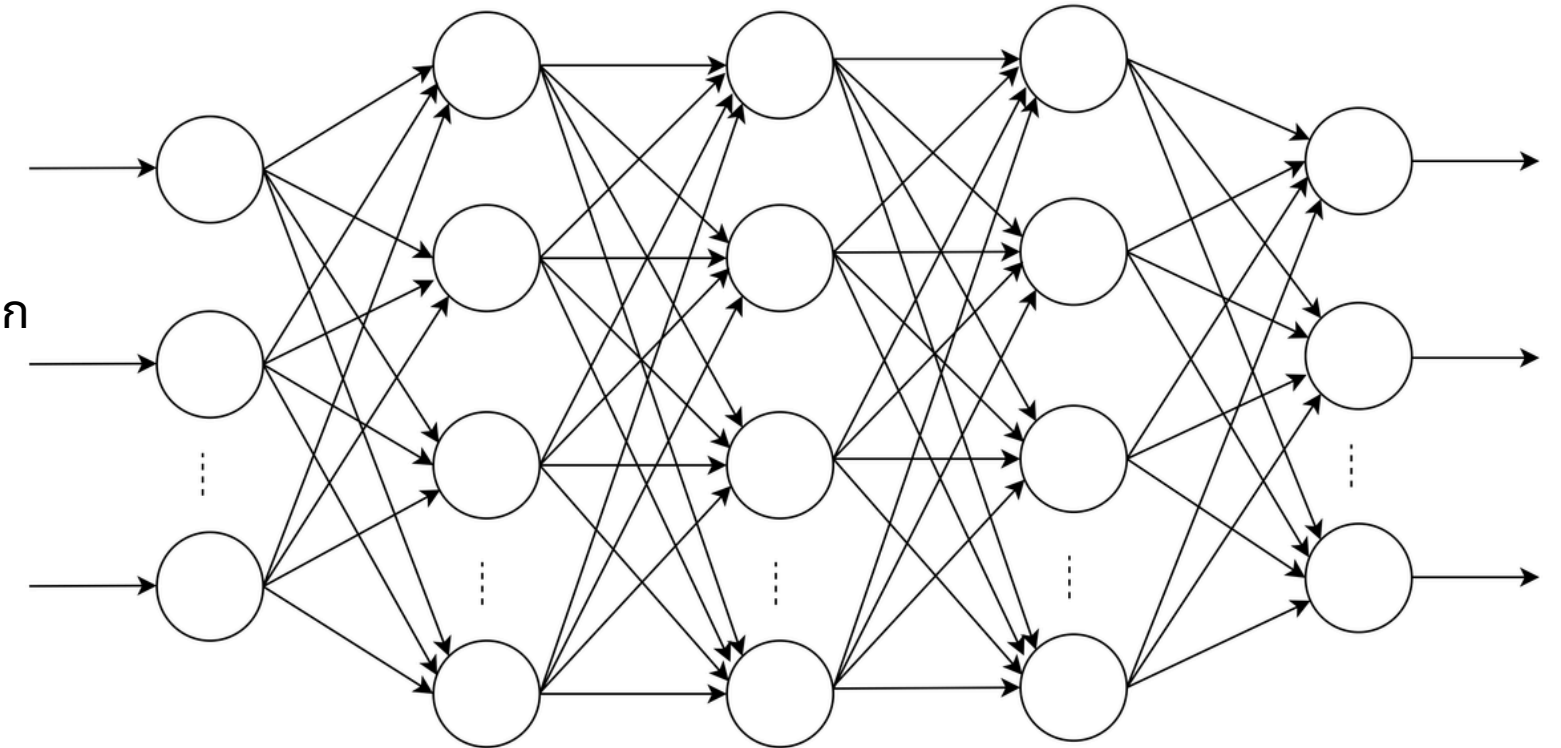
จำนวน Hidden Layer, จำนวน Node ในแต่ละ Layer, Activation Function, Optimizer, Lost function, Learning Rate, Momentum (เฉพาะใน sgd), จำนวน epoch, weight เริ่มต้น

- **การกำหนด จำนวน Hidden Layer**

- ปัญหาไม่ซับซ้อนมากใช้เพียง 1 layer ก็พอ
จำนวน hidden layer ที่มากกว่านี้มักจะไม่เพิ่มความถูกต้อง)
- ปัญหาซับซ้อน (ภาพ เสียง ข้อความ ฯลฯ) แม้จะใช้ MLP ได้
แต่แนะนำให้ใช้ DL อื่น เช่น CNN หรือ LSTM

• การกำหนด จำนวน Node

- Input Layer
 - ถ้าเป็นข้อมูลแบบตาราง หรือ structure data ใช้จำนวน node เท่ากับจำนวน column
 - ถ้าเป็นข้อมูลภาพและเสียง (แปลงเสียงเป็นภาพ) ใช้จำนวน node เท่ากับจำนวน pixel ซึ่งคำนวณมาจากขนาดของภาพ (ขนาดภาพที่นิยม ภาพเล็ก=28×28=784, ภาพกลาง=32×32=1024, ภาพใหญ่=224×224=50176)
 - ถ้าเป็นข้อมูลข้อความ ใช้จำนวน node เท่ากับจำนวนคำ (word) ในประโยค หรือในเอกสาร
- Hidden Layer
 - ถ้า node น้อยเกินไป จะเกิดปัญหา underfitting มากเกินไป จะเกิดปัญหา overfitting
 - ถ้าเป็นข้อมูลแบบตาราง ไม่ควรมากกว่าจำนวนตัวแปรต้น (Input Node) หรืออาจใช้สูตร Hidden node $\approx$ (จำนวน Input Node + จำนวน Output Node) / 2
 - ถ้าเป็นข้อมูลภาพ (ภาพเล็ก 128–512 node, กลาง 256–1024 node, ใหญ่ 1024–4096 node)
 - ถ้าข้อมูลน้อย จำนวน node น้อย ถ้าข้อมูลมาก (เช่น คลิปสั้น/ล้านแถว) จำนวน node มาก
 - ทดลองเริ่มที่ค่าน้อย ๆ ก่อน (อาจเริ่มที่ครึ่งหนึ่งของจำนวนตัวแปรต้น) แล้วค่อย ๆ เพิ่มจำนวน node
- Output Layer
 - ถ้าเป็น regression หรือ binary class ให้ใช้ 1 node
 - ถ้าเป็น multi-class ให้ใช้จำนวน node เท่ากับจำนวน class

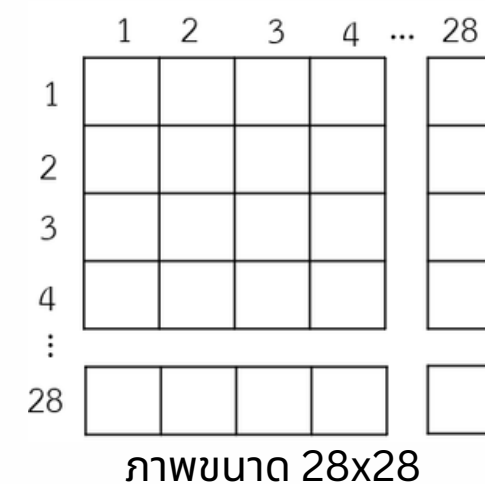


รายการที่รีวิวภาพยนตร์

รีวิวที่	รีวิว	ความรู้สึก
1	หนังเรื่องนี้ดีมาก สนุกสุดๆ	เชิงบวก
2	เป็นหนังที่ดี แต่ตอนจบไม่สนุกเลย	เชิงลบ
3	ฉันไม่ชอบหนังเรื่องนี้เลย	เชิงลบ

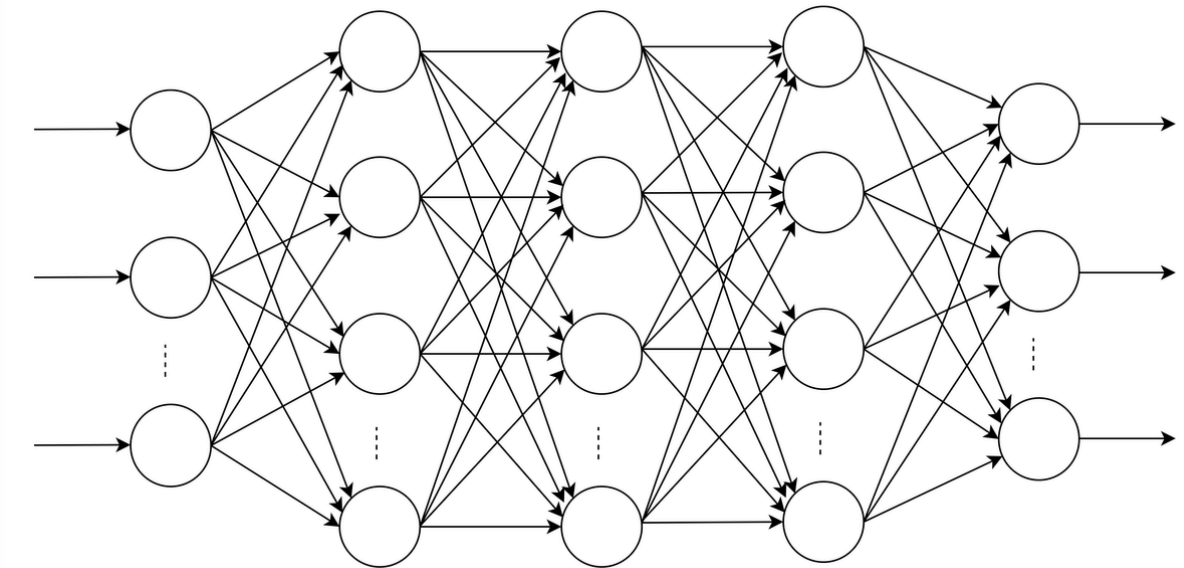
แปลงข้อมูลด้วยวิธี bag of words

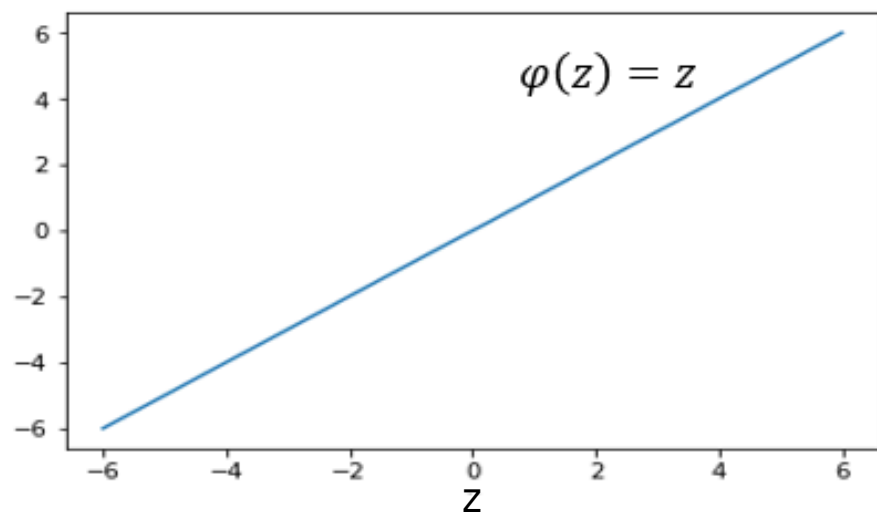
รีวิวที่	หนัง	ดี	มาก	สนุก	สุดๆ	ตอน	จบ	ไม่	ชอบ	เลย	ความรู้สึก
1	1	1	1	1	1	0	0	0	0	0	1
2	1	1	0	1	0	1	1	1	0	1	0
3	1	0	0	0	0	0	0	1	1	1	0



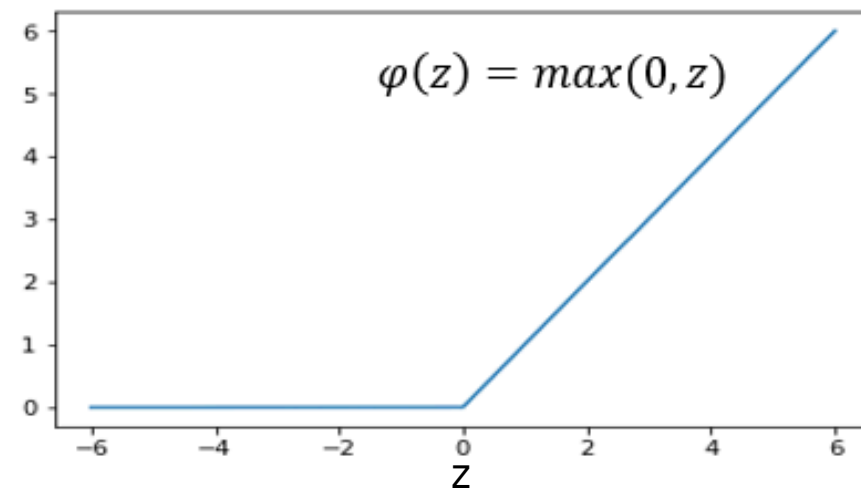
รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

- **การกำหนด Activation Function** เพื่อใช้เป็นตัวกระตุ้นหรือยับยั้งสัญญาณ
 - **Hidden layer**
 - ปัจจุบันนิยมใช้ ReLU เนื่องจากคำนวณได้เร็วกว่า, อดีตนิยมใช้ Sigmoid
 - **Output Layer**
 - Regression ใช้ Linear หรือ ReLU
 - Binary class ใช้ Sigmoid
 - Multi-class ใช้ Softmax

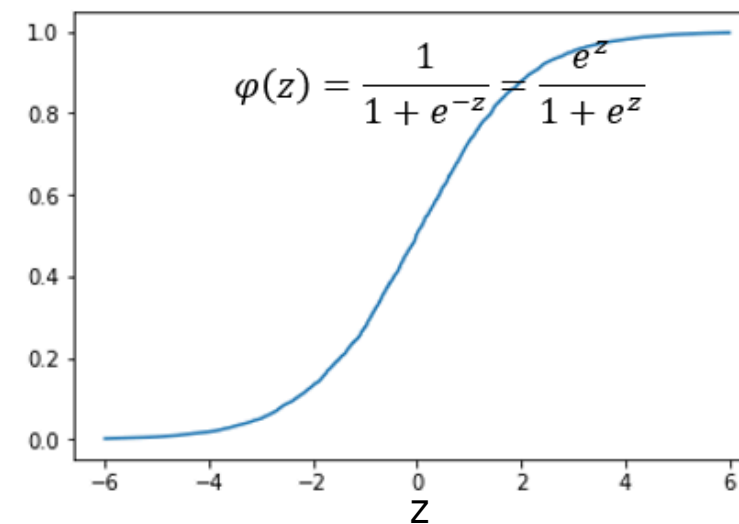




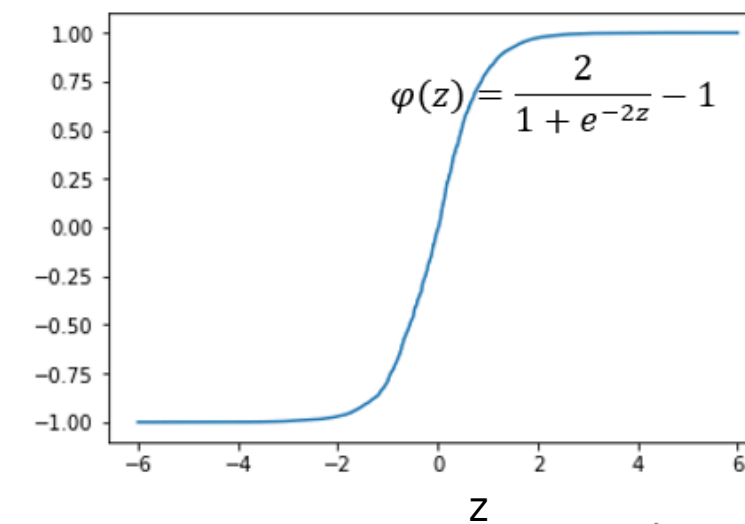
Identity (linear)



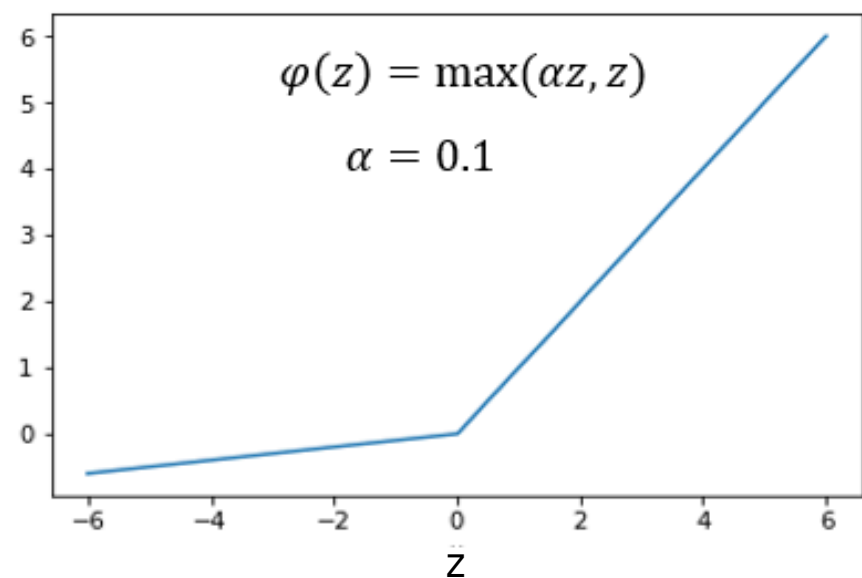
Rectified Linear Unit (ReLU)



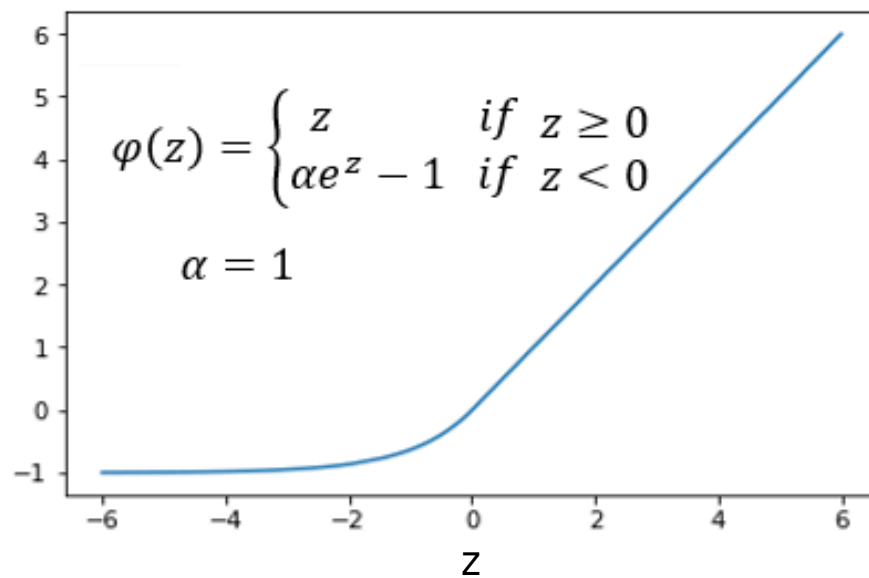
Logistic Sigmoid (logistic)



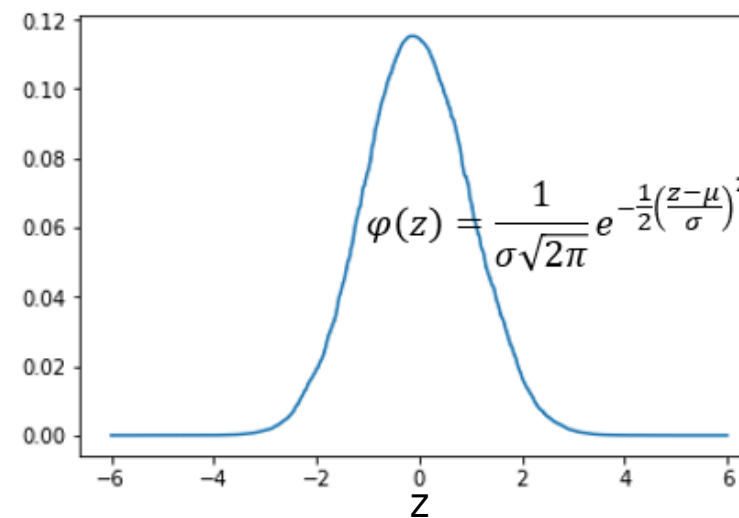
Hyperbolic Tangent Sigmoid (tanh)



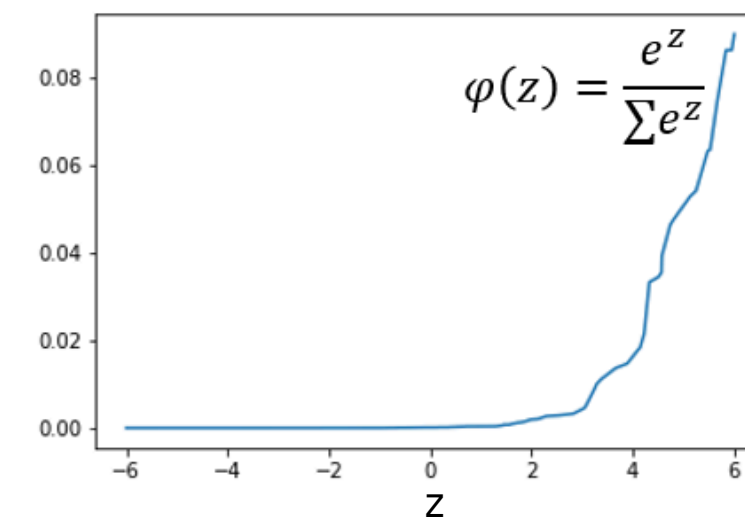
Leaky ReLU



Exponential Linear Unit (ELU)



Gaussian (RBF)



Softmax

ภาพที่ 4-4 : เปรียบเทียบกราฟของ Activation Function

- **การกำหนด Optimizer (Solver)** เพื่อใช้ในการปรับ weight
 - **Gradient Descent (GD)** ใช้ข้อมูลทุก data points เพื่อปรับ weight ใน 1 รอบ เรียกว่าการ Train แบบ offline หรือแบบ batch
 - **Stochastic Gradient Descent (SGD)** ใช้ข้อมูล 1 data point (online) หรือ บาง data points (mini-batch) เพื่อปรับ weight ใน 1 รอบย่อย -- SGD ไม่มี momentum ในตัวแต่เพิ่มได้
 - **Adaptive Moment Estimation (Adam)** เป็นอัลกอริทึมที่พัฒนามาจาก SGD โดยใช้มีการคำนวณค่า momentum ในตัว และการปรับ learning rate อัตโนมัติ (adaptive learning rate) ทำให้การเรียนรู้มีความเสถียรและรวดเร็วขึ้น **(นิยม ใช้ง่าย เสถียร)**
 - **Jimmy Ba Limited-memory Broyden–Fletcher–Goldfarb–Shanno (LBFGS)** เป็นอัลกอริทึมในกลุ่ม quasi-Newton methods ที่ใช้การประมาณ Hessian matrix (เมทริกซ์อนุพันธ์ลำดับสอง) แบบประหยัดหน่วยความจำ เหมาะสำหรับปัญหาที่มีจำนวนพารามิเตอร์มาก

หมายเหตุ : ใน GD การ Train 1 รอบ จะปรับ weight 1 ครั้ง แต่ใน SGD การ Train 1 รอบ จะปรับ weight หลายครั้ง

- **การกำหนด Lost function** เพื่อใช้ในการคำนวณ error
 - **Regression** ใช้ Mean Square Error (MSE)
 - **Binary class** ใช้ Binary Cross-Entropy (BCE)
 - **Multi-class** ใช้ Categorical Cross-Entropy (CE)
- **การกำหนด Learning Rate (η)** เพื่อใช้ในการควบคุมความ เร็ว/ช้า ของการเรียนรู้
 - η สูงเกินไป การเรียนรู้จะแกว่ง (divergence) มาก ถ้าต่ำเกินไปจะเรียนรู้จบ (convergence) ช้า
 - ปกติประมาณ 0.001
 - ถ้า loss ไม่ลด ให้ ลด Learning Rate ถ้า convergence ช้า ให้ เพิ่ม Learning Rate
- **การกำหนด Momentum (μ)** เพื่อใช้ในการควบคุมทิศทางของการเรียนรู้ (รักษาทิศความชันของ gradient)
 - ปกติประมาณ 0.9
 - ค่าประมาณ 0.9-0.99 เคลื่อนที่ราบเรียบ แต่เปลี่ยนทิศช้ากว่า
 - ค่าประมาณ 0.5-0.8 เคลื่อนที่เร็ว แต่แกว่งง่ายกว่า

- **การกำหนด จำนวน epoch** หรือจำนวนรอบของการ train
 - **ขึ้นกับขนาดข้อมูล** ถ้าข้อมูลขนาดใหญ่จำนวนรอบอาจน้อยลง ถ้าข้อมูลขนาดเล็กจำนวนรอบอาจมากขึ้น
 - **ใช้ Early Stopping** โดยดูค่า validation loss หากค่า loss ไม่เปลี่ยนหรือเปลี่ยนน้อยกว่าที่กำหนด การ train ก็จะหยุด
- **การกำหนด Weight เริ่มต้น**
 - เปรียบเสมือนการตั้งค่าความเข้มของสัญญาณตั้งต้น หรือเป็นการให้ความรู้เบื้องต้นแก่โมเดลก่อนการเรียนรู้จริง
 - โดยปกติจะใช้การสุ่ม (random)
 - ห้ามให้ค่าเริ่มต้นเป็นศูนย์ทั้งหมด (จะไม่สามารถเรียนรู้ได้)

ขั้นที่ 1 : คำนวณด้วยสมการ Forward เพื่อให้ได้ผลลัพธ์ หรือผลการทำนาย

$$\mathbf{h} = \varphi(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = \varphi(\mathbf{w}^{(2)}\mathbf{x} + \mathbf{b}^{(2)})$$

ขั้นที่ 2 : คำนวณ Loss Function (Error) เพื่อประเมินประสิทธิภาพของโมเดลในแต่ละรอบการเรียนรู้

- **Regression** (MSE + Linear)

$$\mathcal{L} = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$$

- **Binary class** (BCE + Sigmoid)

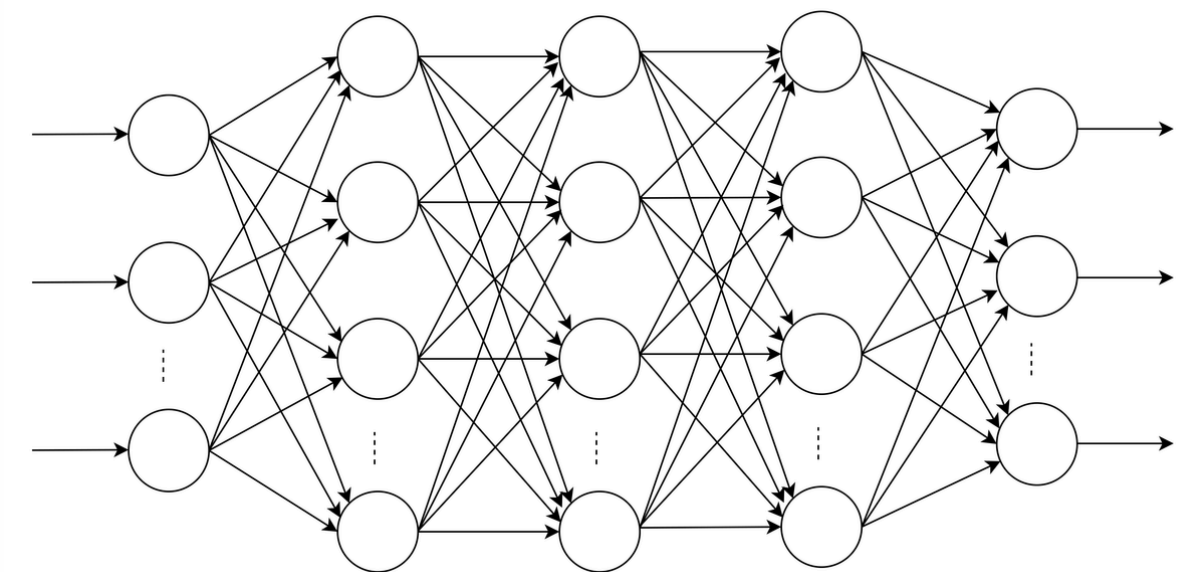
$$\hat{y} = \sigma(z^{(L)}),$$

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- **Multi-class** (CE + Softmax)

$$p = \text{softmax}(z^{(L)}),$$

$$\mathcal{L} = -\sum_k y_k \log p_k$$



ขั้นที่ 3 : ทำ Backpropagation เพื่อย้อนกลับไปเพื่อคำนวณค่า gradient

นิยาม error ที่ขั้นสุดท้าย:

- Softmax + CE: $\delta^{(L)} = p - y$ (สวยและเร็ว)
- Sigmoid + BCE: $\delta^{(L)} = \hat{y} - y$
- ทั่วไป: $\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot f'^{(L)}(z^{(L)})$

ย้อนกลับสำหรับชั้น $l = L - 1, \dots, 1$:

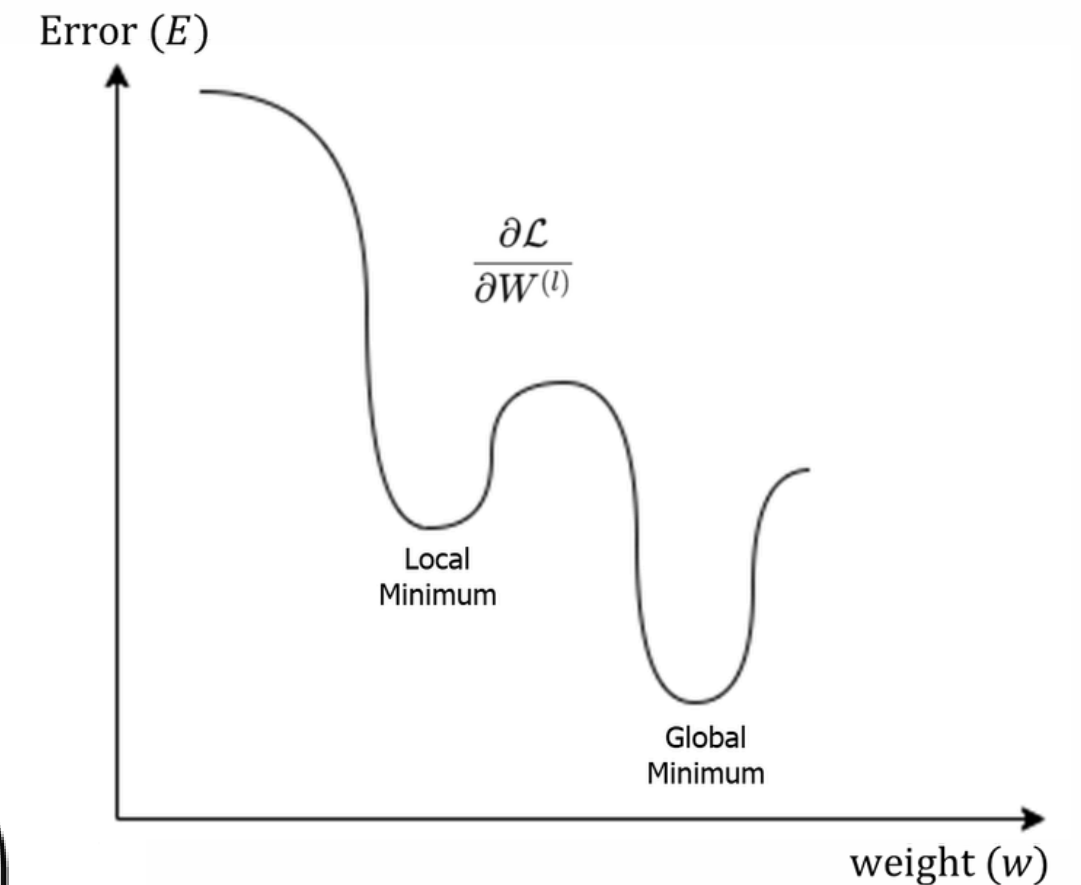
$$\delta^{(l)} = \left(W^{(l+1)}\right)^{\top} \delta^{(l+1)} \odot f'^{(l)}(z^{(l)})$$

เกรเดียนต์ต่อพารามิเตอร์:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^{\top}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

ขั้นที่ 4 : ปรับ Weight โดยใช้ Optimizer ร่วมกับค่า Gradient ที่คำนวณได้

$$\text{SGD: } W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$



ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่า

- ครบจำนวนรอบ (Epoch) ของการ Train ตามที่กำหนดไว้
- Early Stopping โดยดูค่า validation loss หากค่า loss ไม่เปลี่ยนหรือเปลี่ยนน้อยกว่าที่กำหนด การ train ก็จะหยุด

ขั้นตอนการทดสอบ (Test) โมเดล

ในการคำนวณหาผลลัพธ์ ($\hat{y}$)ให้นำเอา Test data เฉพาะส่วนที่เป็นตัวแปรต้น (x) ใส่เข้าไปในสมการที่ได้จากการ Train model โดยมีรูปแบบของสมการ ดังนี้

$$h_j = \varphi(w_{1j}^{(1)}x_1 + \dots + w_{mj}^{(1)}x_m + b_j^{(1)})$$

h_j คือ hidden node ที่ j

φ คือ activation function (เช่น ReLU หรือ Sigmoid)

$w_{1j}^{(1)}$ คือ ค่า weight ที่เชื่อมจาก input node ที่ 1 ไปยัง hidden node ที่ j

x_1 คือ ข้อมูล input node ที่ 1 ซึ่งมีทั้งหมด m โหนด

$b_j^{(1)}$ คือ ค่า bias ของ hidden node ที่ j

$$\hat{y}_k = \varphi(w_{1k}^{(2)}h_1 + \dots + w_{jk}^{(2)}h_j + w_{pk}^{(2)}h_p + b_k^{(2)})$$

$\hat{y}_k$ คือ output node ตัวที่ k

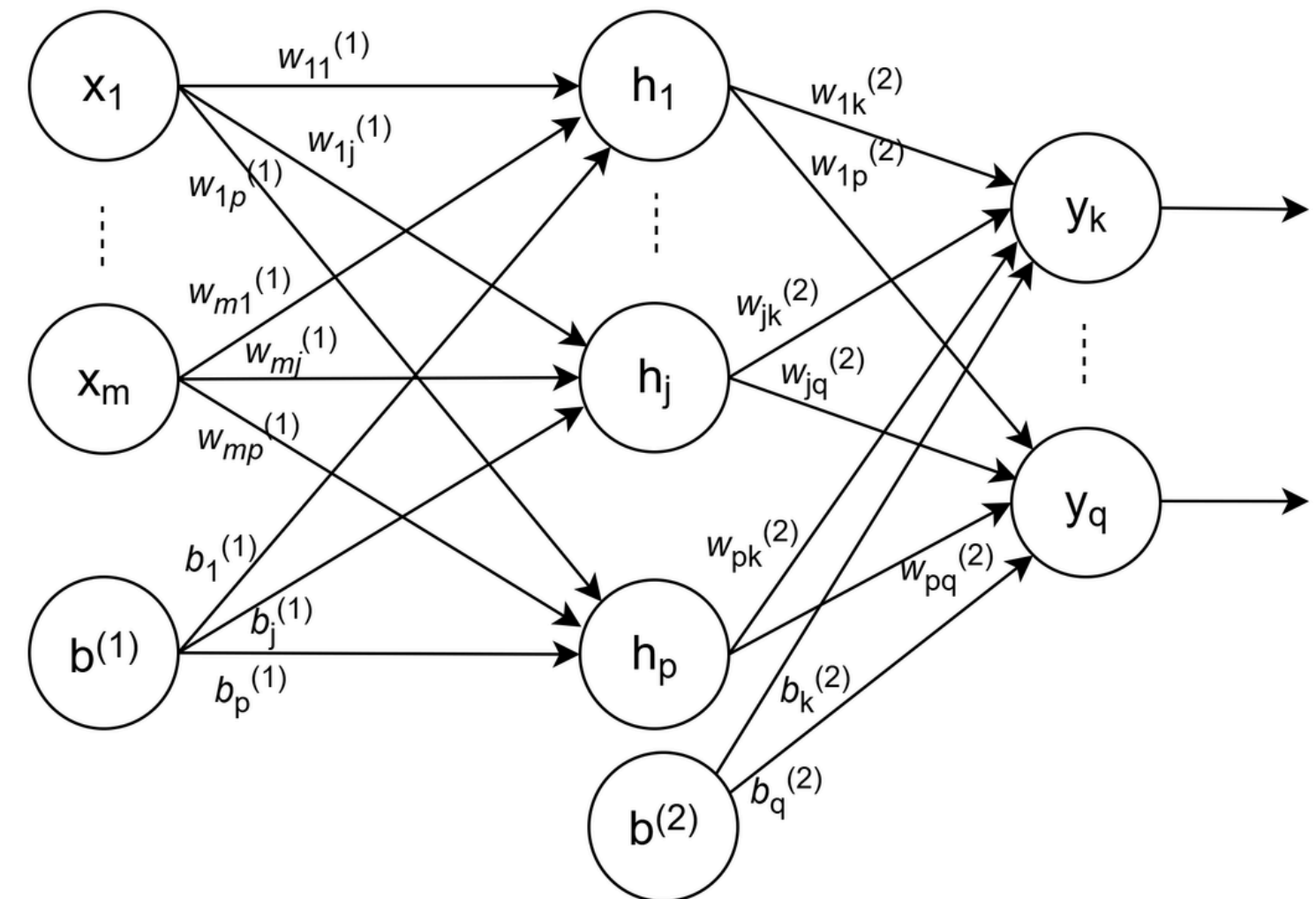
φ คือ activation function เช่น Sigmoid สำหรับ binary class,

Softmax สำหรับ multi-class, และ linear/ReLU สำหรับ regression

$w_{1k}^{(2)}$ คือ ค่า weight ที่เชื่อมจาก hidden node ที่ 1 ไปยัง output node ที่ k

h_1 คือ hidden node ที่ 1 ซึ่งมีทั้งหมด p โหนด

$b_k^{(2)}$ คือ ค่า bias ของ output node ที่ k



4.3.3 ตัวอย่างการคำนวณ MLP

ข้อที่ 1 : จงแสดงวิธีการคำนวณ MLP เพื่อประเมินราคาที่ดิน
กำหนดให้

- Input Node = 2
- Hidden Node = 3 (Logistic Sigmoid)
- Output Node = 1 (Linear)
- Hidden Weights (Input → Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.570 & 0.283 & 0.253 \\ 0.065 & -0.095 & 0.197 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.076, 0.496, 0.589]$
- Output Weight (Hidden → Output): $\mathbf{W}_y = \begin{bmatrix} 49.361 \\ 49.991 \\ 49.618 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [49.675]$

ตารางที่ 4-1 : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	220	2	?

วิธีทำ

- คำนวณใน Hidden Layer

$$h_1 = \varphi(0.570 \times 220 + 0.065 \times 2 - 0.076) = \varphi(125.454)$$

$$= \frac{1}{1 + e^{-125.454}} = 1$$

$$h_2 = \varphi(0.283 \times 220 + (-0.095) \times 2 + 0.496) = \varphi(62.566)$$

$$= \frac{1}{1 + e^{-62.566}} = 1$$

$$h_3 = \varphi(0.253 \times 220 + (0.197) \times 2 + 0.589) = \varphi(56.643)$$

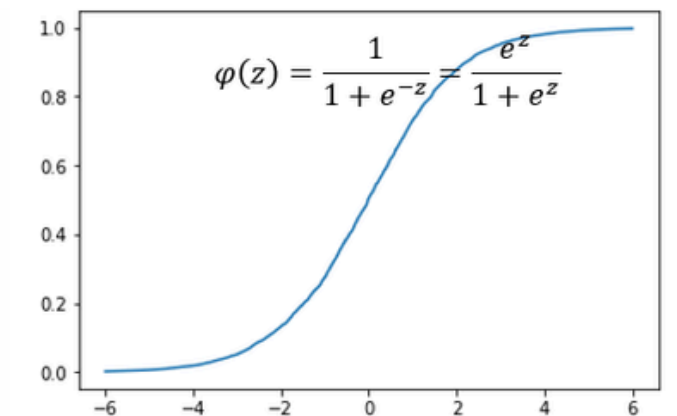
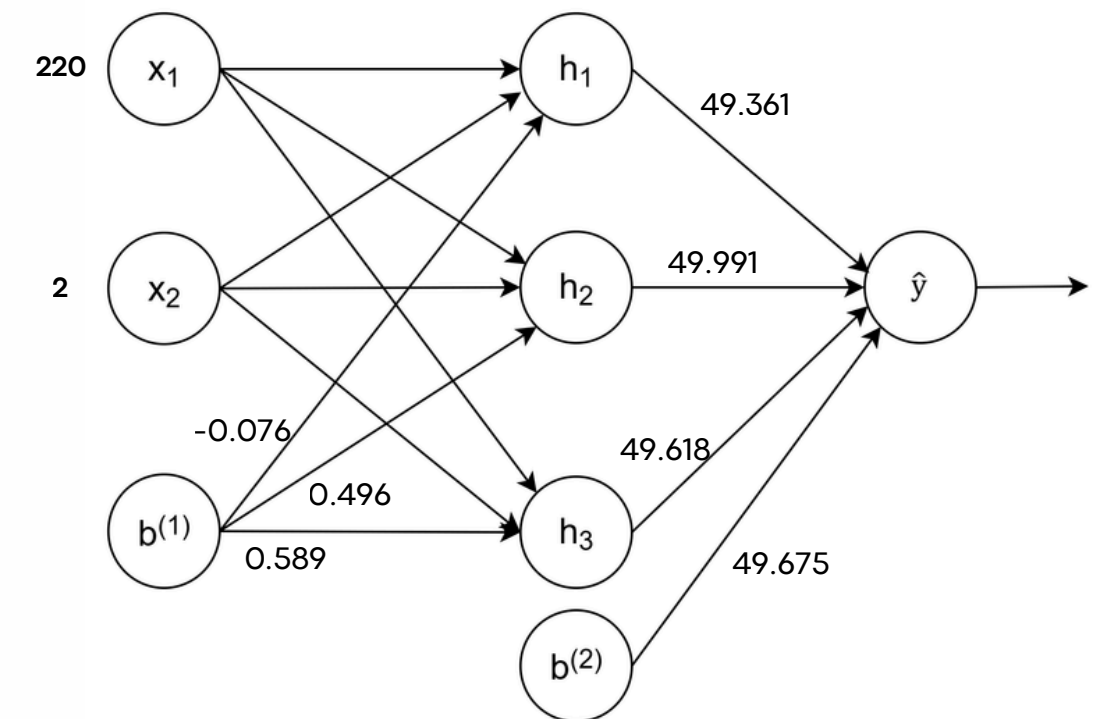
$$= \frac{1}{1 + e^{-56.643}} = 1$$

- คำนวณใน Output Layer

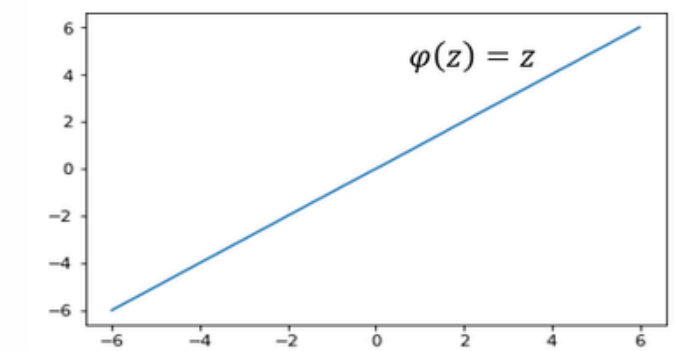
$$\hat{y} = \varphi(49.361 \times 1 + 49.991 \times 1 + 49.618 \times 1 + 49.675) = \varphi(198.645) = 198.645$$

- คูณ 1,000 เพื่อปรับให้อยู่ในหน่วย บาท/ตร.ว. จะได้
- $$198.645 \times 1,000 = 198,645$$

ตอบ ราคาที่ดินที่ประเมินได้ คือ 198,645 บาท/ตร.ว.



Logistic Sigmoid (logistic)



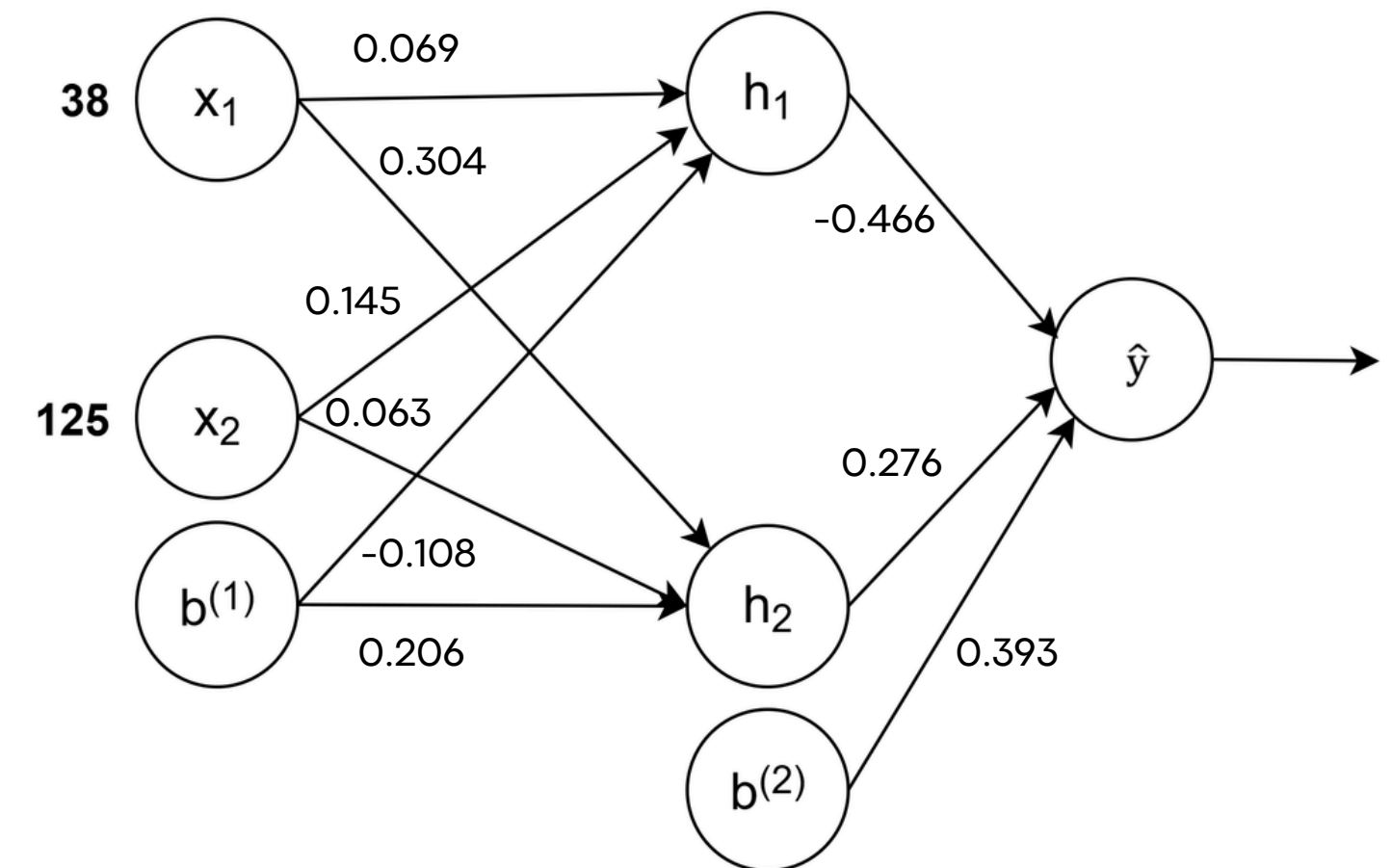
Identity (linear)

ข้อที่ 2 : จงแสดงวิธีการคำนวณ MLP เพื่อวินิจฉัยการป่วยเป็นโรคเบาหวาน กำหนดให้

- Input Node = 2
- Hidden Node = 2 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.069 & 0.304 \\ 0.145 & 0.063 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.108, 0.206]$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = \begin{bmatrix} -0.466 \\ 0.276 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [0.393]$

ตารางที่ 4-2 : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาล ในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



วิธีทำ

จาก

$$h_1 = \varphi(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + b_1^{(1)})$$

$$h_2 = \varphi(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)})$$

$$\hat{y}_1 = \varphi(w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + b_1^{(2)})$$

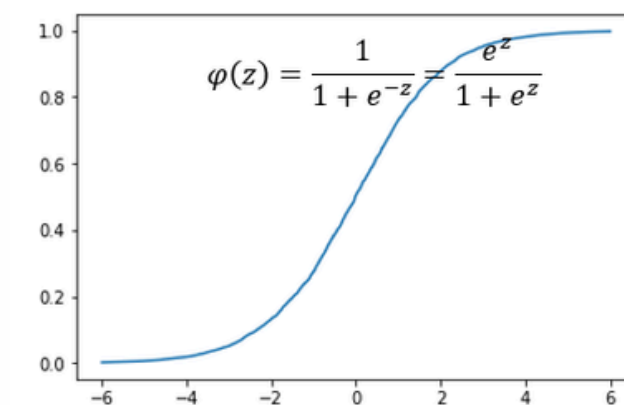
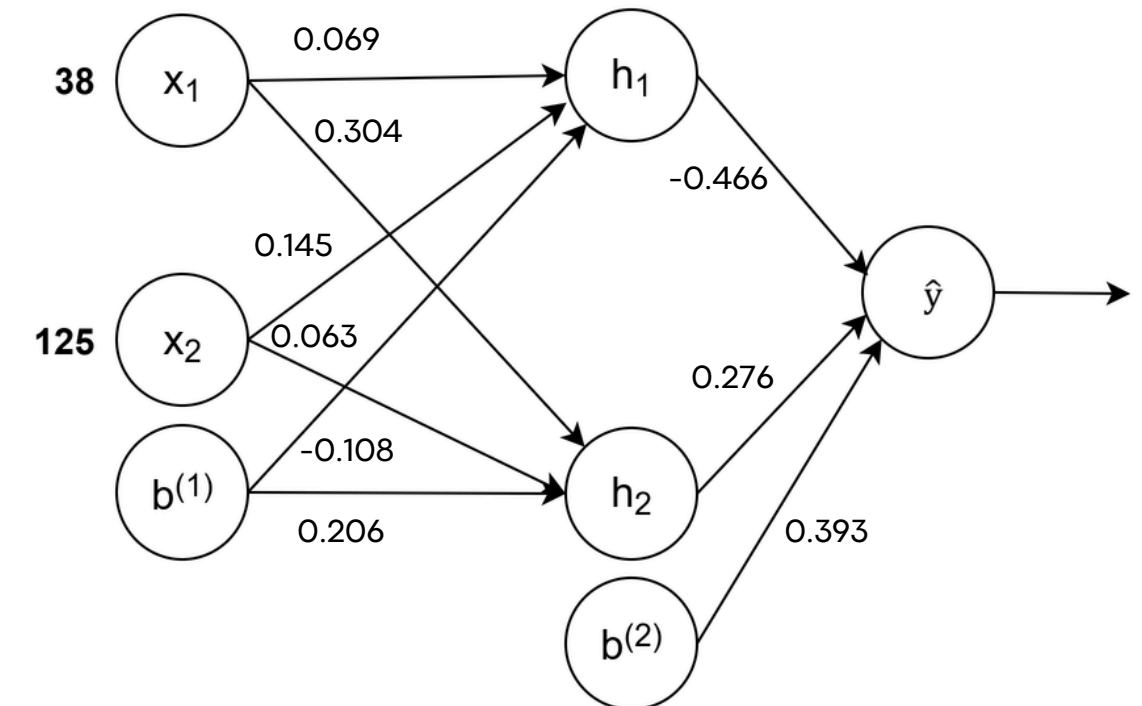
จะได้

$$\begin{aligned} h_1 &= \varphi(0.069 \times 38 + 0.145 \times 125 - 0.108) = \varphi(20.639) \\ &= \frac{1}{1 + e^{-20.639}} = 1 \end{aligned}$$

$$\begin{aligned} h_2 &= \varphi(0.304 \times 38 + 0.063 \times 125 + 0.206) = \varphi(19.633) \\ &= \frac{1}{1 + e^{-19.633}} = 1 \end{aligned}$$

$$\begin{aligned} \hat{y} &= \varphi(-0.466 \times 1 + 0.276 \times 1 + 0.393) = \varphi(0.203) \\ &= \frac{1}{1 + e^{-0.203}} = 0.55 \end{aligned}$$

เนื่องจากค่า prob ที่ได้มากกว่า 0.5 ทำให้ class label = 1
ดังนั้น ผู้ที่เข้ารับการตรวจโรค เป็นโรคเบาหวาน

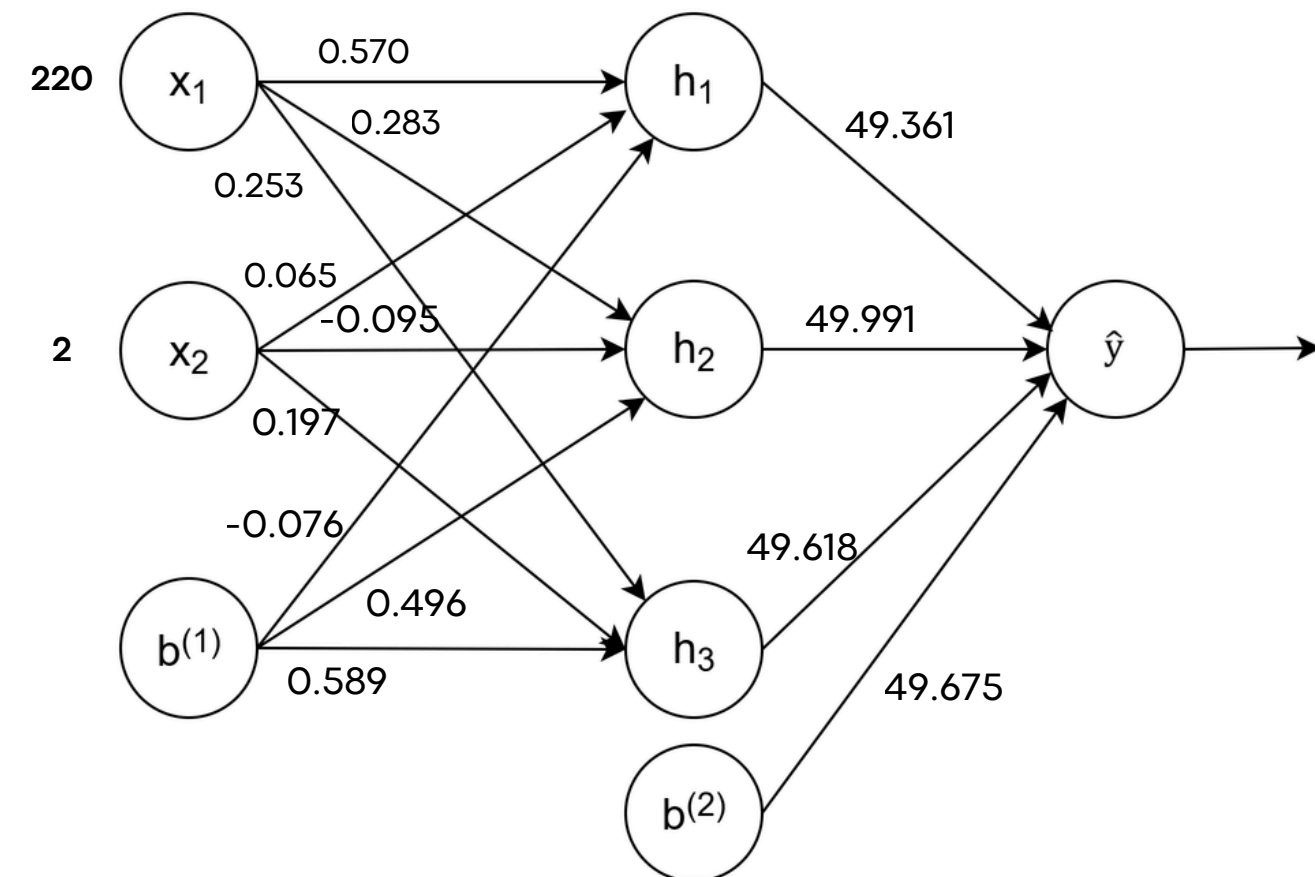


Logistic Sigmoid (logistic)

4.3.4 ตัวอย่างโค้ด MLP

สำหรับข้อที่ 1

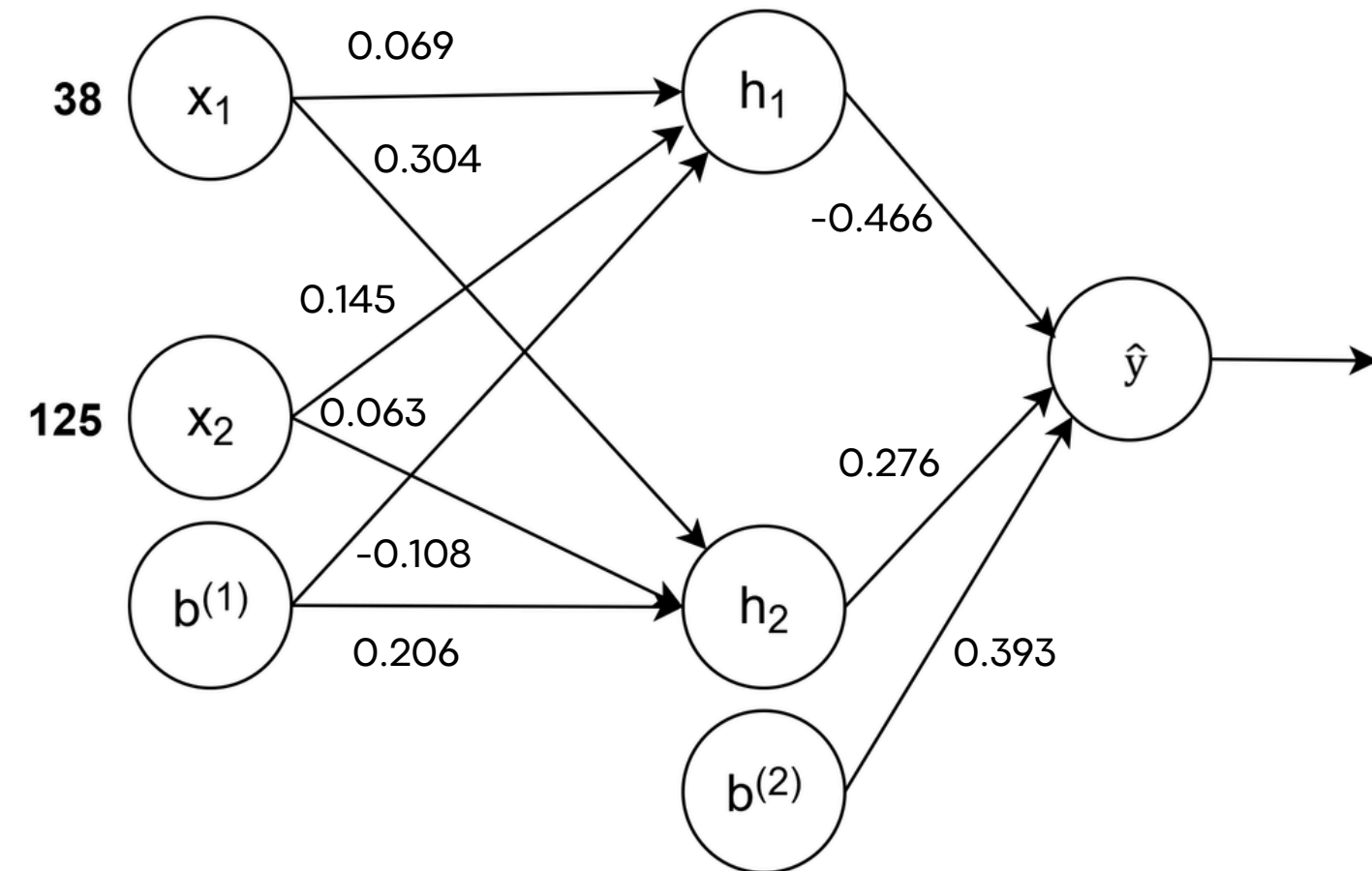
```
1  #Import Python Library
2  from sklearn.neural_network import MLPRegressor
3
4  #Load Data
5  x_train = [[50, 5],[100, 4], [80, 5], [200, 3], [500, 1],
6             [300, 2], [700, 0], [600, 1], [250, 3],[120, 4]]
7  y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
8  x_test = [[220, 2]]
9
10 #Build Model
11 net = MLPRegressor(
12     hidden_layer_sizes=(3),
13     activation='logistic',#sigmoid function
14     solver='sgd',
15     batch_size=5,
16     learning_rate_init=0.001,
17     max_iter=250,
18     random_state=0)
19
20 net.fit(x_train,y_train)
21 print(f'weight: {net.coefs_}') #show weight
22 print(f'bias: {net.intercepts_}') #show bias
23
24 #Test Model
25 y_pred = net.predict(x_test)
26 print(f'ราคาประเมิน คือ {y_pred[0]*1000:,.0f} บาท/ตร.ว.')
```



```
weight: [array([[ 0.56959927,  0.28340219,  0.25298845],
                 [ 0.06548977, -0.09544737,  0.19674558]]), array([[49.36103808],
                 [49.99053636],
                 [49.61814864]])]
bias: [array([-0.07585905,  0.49578219,  0.58893365]), array([49.67477853])]
ราคาประเมิน คือ 198,645 บาท/ตร.ว.
```

สำหรับข้อที่ 2

```
1 #Import Python Library
2 from sklearn.neural_network import MLPClassifier
3 import pandas as pd
4 import numpy as np
5
6 #Load Data
7 / df = pd.read_csv('data/diabetes.csv')
8 x_train = df[['age', 'blood_glucose']].to_numpy()
9 y_train = df['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
10 x_test = np.array([[38, 125]])
11
12 #Build Model
13 net = MLPClassifier(
14     hidden_layer_sizes=(2),
15     activation='logistic',
16     solver='sgd',
17     batch_size=len(x_train),
18     learning_rate_init=0.001,
19     max_iter=250,
20     random_state=0)
21
22 net.fit(x_train,y_train)
23 print(f'weight: {net.coefs_}') #show weight
24 print(f'bias: {net.intercepts_}') #show bias
25
26 #Test Model
27 prop = net.predict_proba(x_test)
28 print(f'probability: {prop[0][1]*100:.2f}%')
29 y_pred = net.predict(x_test)
30 print(f'Class Label: {y_pred[0]}') #show class label
```



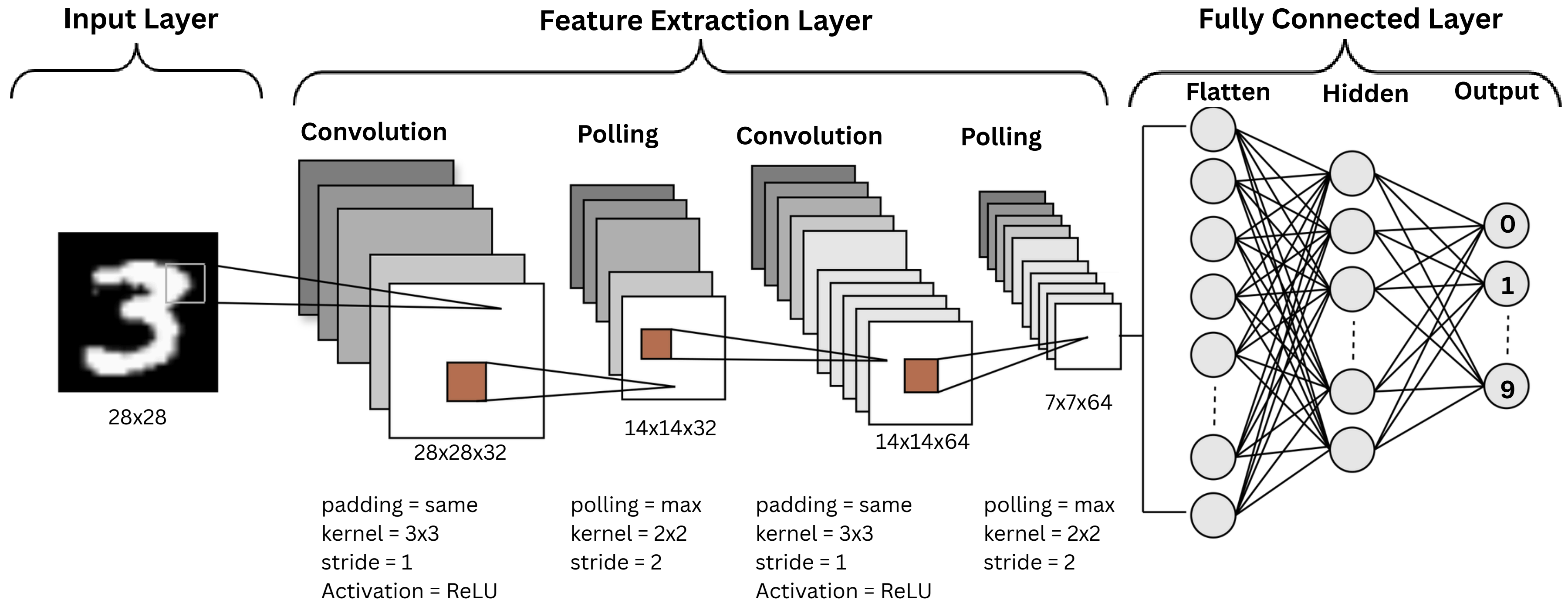
```
weight: [array([[0.06903344, 0.30430758],
                [0.14533409, 0.06344131]]), array([[ -0.46582199],
                [ 0.27584295]])]
bias: [array([ -0.10796832,  0.20632507]), array([0.39324777])]
probability: 55.06%
Class Label: 1
```

4.4 คอนโวลูชันนัล นิวรัลเน็ตเวิร์ก (CNN)

คอนโวลูชันนัล นิวรัลเน็ตเวิร์ก (Convolutional Neural Network: CNN)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Feed forward neural network
รับข้อมูลเข้ามา (Input เช่น รูปภาพ) แล้วส่งผ่านชั้นต่าง ๆ เช่น Convolution Layer, Pooling Layer, Fully Connected Layer
- เรียนรู้โดยใช้ Backpropagation
ปรับปรุงความรู้ในทิศทางย้อนกลับ โดยใช้ error เพื่อคำนวณหาน้ำหนัก (Weight) ของฟิลเตอร์ (Filter/Kernel) และชั้น Fully Connected
- มีคุณสมบัติในการดึงคุณลักษณะ (Feature Extraction) อัตโนมัติ
ชั้น Convolution และ Pooling ช่วยดึงคุณลักษณะเชิงพื้นที่ (Spatial Features) จากข้อมูล เช่น ขอบ เส้น รูปร่าง และลักษณะซับซ้อนขึ้นเรื่อย ๆ
- นิยมใช้ในงานประมวลผลภาพและคอมพิวเตอร์วิชัน
เช่น Image Segmentation, Image Classification, Object Detection, Face Recognition

4.4.1 โครงสร้างของ CNN



ภาพที่ 4-5 : โครงสร้างของ CNN

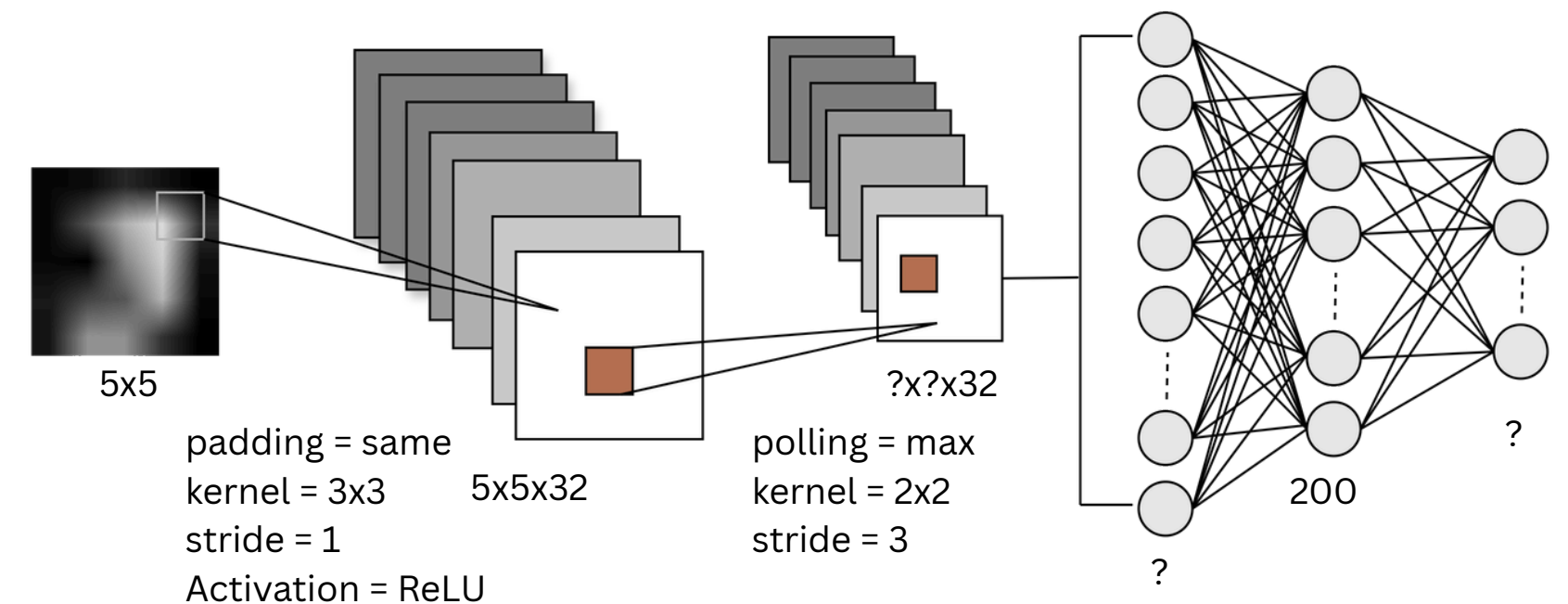
4.4.2 ตัวอย่างการคำนวณ CNN

กำหนดให้โครงสร้างของ CNN สำหรับทำนายตัวเลข 0-9 ดังนี้

- **Input Layer:** ใช้ภาพขาวดำ ขนาด 5×5 pixels (ใช้เลข 3 เป็นตัวอย่างการคำนวณ)
- **Feature Extraction Layer:** มี 1 convolution และ 1 polling layer ดังนี้
 - **Convolution:** มี 32 filter แต่ละ filter มีขนาด 3×3 pixel ดังแสดงในรูป, stride เท่ากับ 1 และมีการ padding เพื่อให้ได้ภาพขนาดเท่าเดิม, activation function คือ ReLU
 - **Pooling Layer:** ใช้ฟังก์ชัน max, ใช้จำนวน filter เท่าเดิม แต่ละ filter มีขนาด 2×2 pixel, stride เท่ากับ 3, ไม่มีการ padding
- **Fully Connected Layer:** มี 1 hidden layer
 - **Flatten Layer:** มีจำนวน node คำนวณมาจาก Layer ก่อนหน้า
 - **Hidden Layer:** มีจำนวน node เท่ากับ 200
 - **Output Layer:** มีจำนวน node เท่ากับ จำนวนตัวเลข 0-9

0	0	1
0	1	0
1	0	0

filter 3x3



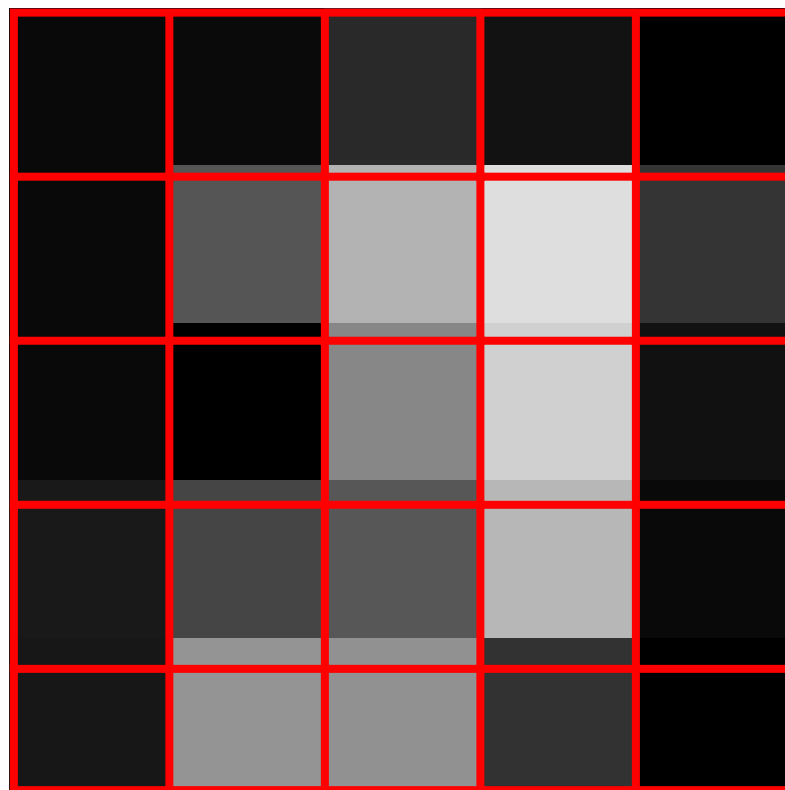
จงตอบคำถามต่อไปนี้

1. แสดงการคำนวณเพื่อหา feature map ใน convolution layer (ให้แสดงการคำนวณ ทั้งแบบไม่มี padding และมี padding ด้วย เพื่อดูความแตกต่าง)
2. แสดงการคำนวณเพื่อหา feature map ใน polling Layer
3. แสดงการคำนวณหาจำนวน node ใน flatten layer
4. แสดงการคำนวณหาจำนวน weight ระหว่าง flatten layer กับ hidden layer
5. แสดงการคำนวณหาจำนวน node ใน output Layer
6. แสดงการคำนวณหาจำนวน weight ระหว่าง hidden layer กับ output layer

- **Input Layer**

เป็น Layer แรกของโครงสร้าง CNN มีจำนวน input nodes เท่ากับจำนวน pixels ของภาพ
ดังนั้น มีจำนวน input nodes = 25 (5x5)

- **หาค่าสีของภาพ**



ภาพขนาด 5x5

หาค่าสีของภาพ
[0-255]

9	23	65	51	9
9	74	172	218	52
13	18	129	202	22
25	90	99	151	13
23	148	145	48	2

ภาพขนาด 5x5

หมายเหตุ : ภาพดังกล่าวเป็นภาพของเลข 3 มีขนาด 5x5 ซึ่งทำขึ้นมาเพื่อแสดงตัวอย่างการคำนวณเท่านั้น
ในสถานการณ์จริงควรใช้ขนาดของภาพที่ใหญ่กว่านี้

- ทำ Normalization หรือ Standardization

สูตรการทำ standardization

$$\hat{x} = \frac{x - \mu}{\sigma}$$

เมื่อ $\hat{x}$ คือ ค่าสีใหม่ μ คือ ค่าเฉลี่ยของค่าสี
 x คือ ค่าสีเดิม σ คือ ส่วนเบี่ยงเบนมาตรฐานของค่าสี

สูตรการ normalization ให้อยู่ในช่วง 0-1

$$\hat{x} = \frac{x - \min}{\max - \min} \quad \text{หรือ} \quad = \frac{x}{\max}$$

เมื่อ $\min$ คือ ค่าต่ำสุดของค่าสี นิยมกำหนดเป็น 0
 $\max$ คือ ค่าสูงสุดของค่าสี นิยมกำหนดเป็น 255

ตัวอย่างการคำนวณ เมื่อ x เท่ากับ 9 จะได้

$$\hat{x} = \frac{x}{\max} = \frac{9}{255} = \frac{9}{255} = 0.035$$

9	23	65	51	9
9	74	172	218	52
13	18	129	202	22
25	90	99	151	13
23	148	145	48	2

ผลการ normalization
โดยใช้สูตร

$$\hat{x} = \frac{x}{\max}$$

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

- **Convolution Layer**

เป็น Layer ที่ดึงคุณลักษณะ (องค์ประกอบ) ของภาพออกมา โดยใช้ Filter หรือ Kernel จากโจทย์ กำหนดให้ filter มีขนาด 3x3 และมีค่าดังรูป (ค่าใน filter ก็คือ weight นั้นเอง)

0	0	1
0	1	0
1	0	0

filter 3x3

- **คำนวณโดยไม่ทำ padding**

- ภาพใหม่ที่ได้ (Feature Map) จะมีขนาดเล็กลง
- สามารถคำนวณหาขนาดของภาพใหม่ (Feature Map Size หรือ Output Size) ดังนี้

$$O = \frac{I - F + 2P}{S} + 1$$

เมื่อ O คือ Output Size
I คือ Input Size
F คือ Filter Size
P คือ Padding Size
S คือ Stride Size

ตัวอย่างการคำนวณหา Output Size เมื่อ I = 5, F = 3, P = 0 (กรณีไม่ทำ Padding), S = 1 จะได้

$$O = \frac{5 - 3 + 2(0)}{1} + 1 = 3$$

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

- ในการคำนวณ จะวาง Filter ทาบลงบนภาพ แล้วเลื่อน Filter จากซ้ายไปขวา และจากบนลงล่างตามค่า Stride
- ถ้าค่า Stride=1 จะเลื่อน Filter ไปทีละ 1 pixel

0	0	1
0	1	0
1	0	0

filter size = 3x3

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.051*1 + 0.29*1 + 0.255*1 = 0.596$$

0.596		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.071 \times 1 + 0.675 \times 1 + 0.2 \times 1 = 0.946$

0.596	0.946	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 3

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.506*1 + 0.855*1 + 0.035*1 = 1.396$$

0.596	0.946	1.396

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 4

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.098*1 + 0.071*1 + 0.675*1 = 0.844$$

0.596	0.946	1.396
0.844		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 5

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.353*1 + 0.506*1 + 0.855*1 = 1.714$$

0.596	0.946	1.396
0.844	1.714	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 6

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.388*1 + 0.792*1 + 0.204*1 = 1.384$

0.596	0.946	1.396
0.844	1.714	1.384

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 7

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.09*1 + 0.353*1 + 0.506*1 = 0.949$$

0.596	0.946	1.396
0.844	1.714	1.384
0.949		

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 8

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$0.58 \times 1 + 0.388 \times 1 + 0.792 \times 1 = 1.76$

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	

Output Size = 3x3

เลื่อน Filter แล้วคำนวณครั้งที่ 9

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

$$0.569*1 + 0.592*1 + 0.086*1 = 1.247$$

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	1.247

Output Size = 3x3

- **คำนวณโดยทำ padding**

- ภาพใหม่ที่ได้ (Feature Map) จะมีขนาดเท่าเดิม
- สามารถคำนวณหา Padding Size ดังนี้

จากสมการเดิม $O = \frac{I - F + 2P}{S} + 1$

ให้ $O = I$ แล้วกลับสมการหาค่า P ได้ดังนี้

$$P = \frac{(I - 1)S - I + F}{2} = \frac{(5 - 1)1 - 5 + 3}{2} = 1 \quad \text{จะได้ Padding Size} = 1$$

0.035	0.09	0.255	0.2	0.035
0.035	0.29	0.675	0.855	0.204
0.051	0.071	0.506	0.792	0.086
0.098	0.353	0.388	0.592	0.051
0.09	0.58	0.569	0.188	0.008

Input Size = 5x5

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$0*1 + 0.035*1 + 0*1 = 0.035$

0.035				

Output Size = 5x5

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$0.035 \times 1 + 0.09 \times 1 + 0 \times 1 = 0.125$

0.035	0.125			

Output Size = 5x5

ทำการคำนวณไปจนถึงรอบที่ 25

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 25 จะได้ Feature Map รอบสุดท้าย ดังรูปด้านขวา

0	0	0	0	0	0	0
0	0.035	0.09	0.255	0.2	0.035	0
0	0.035	0.29	0.675	0.855	0.204	0
0	0.051	0.071	0.506	0.792	0.086	0
0	0.098	0.353	0.388	0.592	0.051	0
0	0.09	0.58	0.569	0.188	0.008	0
0	0	0	0	0	0	0

Input Size + 1 Padding = 7x7

$$0*1 + 0.008*1 + 0*1 = 0.008$$

Feature Map

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Output Size = 5x5

การคำนวณใน Convolution Layer มีทั้งหมด 32 filters ดังนั้น จะต้องคำนวณในลักษณะเดียวกันอีก 31 filters ซึ่ง filter แต่ละตัวจะแตกต่างกันไป เช่น

0	0	1
-1	1	-1
1	0	0

filter 3x3

1	0	-1
0	1	0
-1	0	1

filter 3x3

.....

0	-0.2	0.3
0.2	0.1	0.4
0.2	-0.1	0

filter 3x3

- **คำนวณ Activation Function**

จากโจทย์ กำหนดให้ activation function คือฟังก์ชัน ReLU นั่นคือ

$$\varphi(z) = \max(0, z)$$

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map **ก่อน**คำนวณ
Activation Function

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map **หลัง**คำนวณ
Activation Function

เนื่องจาก Feature Map ก่อนคำนวณ มีค่าเป็นบวกหมด ทำให้ Feature Map หลังคำนวณ มีค่าเท่าเดิม

• Polling Layer

- เป็นการย่อขนาดของ feature map โดยการเก็บเฉพาะค่าที่สำคัญของ feature ไว้
- ทำให้คำนวณเร็วขึ้น เนื่องจากภาพมีขนาดเล็กลง
- ลดการ overfitting เนื่องจาก โมเดลจะไม่จำรายละเอียดเกินไป

จากโจทย์ กำหนดให้ ใช้ฟังก์ชัน max, filter มีขนาด 2×2 pixel, stride เท่ากับ 3, ไม่มีการ padding

คำนวณหา Output Size ได้จาก $O = \frac{I - F + 2P}{S} + 1 = \frac{5 - 2 + 2(0)}{3} + 1 = 2$

เริ่มต้นวาง Filter ทาบลงบนภาพ แล้วทำการคำนวณครั้งที่ 1

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 2

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 3

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396
0.968	

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

เลื่อน Filter แล้วคำนวณครั้งที่ 4

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Feature Map ขนาด 5x5 หลัง
คำนวณ Activation Function

0.596	1.396
0.968	1.247

Feature Map ขนาด 2x2 หลัง
คำนวณ Max Pooling

การคำนวณใน Pooling Layer มีทั้งหมด 32 filters เช่นเดียวกับ Convolution Layer ดังนั้น จะต้องคำนวณในลักษณะเดียวกันอีก 31 filters

- **Flatten Layer**

- เป็นการแปลงข้อมูลจากภาพหรือ Feature Map หลายมิติ ไปเป็นข้อมูล 1 มิติ
- เป็น Layer แรก ภายใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน node ใน flatten layer คือ

จำนวน Flatten Nodes = Width x Height x Depth

เมื่อ Width คือ ความกว้างของ Feature Map

Height คือ ความสูงของ Feature Map

Depth คือ จำนวน Filters

ดังนั้น

$$\text{จำนวน Flatten Nodes} = 2 \times 2 \times 32 = 128$$

- **Hidden Layer**

- เป็นการตีความว่า feature ไหนสำคัญต่อการจำแนก
- อยู่ระหว่าง Flatten Layer และ Output Layer ใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน weight ระหว่าง flatten layer กับ hidden layer

จำนวน Weights = (จำนวน Flatten Nodes + 1) x จำนวน Hidden Nodes

เมื่อ จำนวน Hidden Nodes เท่ากับ 200

ดังนั้น

$$\text{จำนวน Weights} = (128+1) \times 200 = 25,800$$

- **Output Layer**

- เป็นการจำแนกภาพหรือกลุ่มของข้อมูล
- เป็น Layer สุดท้าย ใน Fully Connected Layer

สูตรในการคำนวณหาจำนวน weight ระหว่าง hidden layer กับ output layer

จำนวน Weights = (จำนวน Hidden Nodes + 1) x จำนวน Output Nodes

เมื่อ จำนวน Hidden Nodes เท่ากับ 200

จำนวน Output Nodes เท่ากับ 10 (0-9)

ดังนั้น

$$\text{จำนวน Weights} = (200+1) \times 10 = 2,010$$

สรุปคำตอบ

1. Feature Map ที่ได้จากการไม่ทำ Padding และ ทำ Padding

- Feature Map ที่ได้จากการไม่ทำ Padding เป็นดังนี้

0.596	0.946	1.396
0.844	1.714	1.384
0.949	1.76	1.247

Output Size = 3x3

- Feature Map ที่ได้จากการทำ Padding เป็นดังนี้

0.035	0.125	0.545	0.875	0.89
0.125	0.596	0.946	1.396	0.996
0.341	0.844	1.714	1.384	0.678
0.169	0.949	1.76	1.247	0.239
0.443	0.968	1.161	0.239	0.008

Output Size = 5x5

2. Feature Map ใน Polling Layer

0.596	1.396
0.968	1.247

Output Size = 2x2

3. จำนวน Node ใน Flatten Layer เท่ากับ 128 โหนด
4. จำนวน Weight ระหว่าง Flatten Layer กับ Hidden Layer เท่ากับ 25,800
5. จำนวน Node ใน Output Layer เท่ากับ 10 โหนด
6. จำนวน Weight ระหว่าง Hidden Layer กับ Output Layer เท่ากับ 2,010

4.4.3 ตัวอย่างโค้ด CNN

ติดตั้ง Libraries

```
1 #ติดตั้ง TensorFlow สำหรับการสร้างโมเดล Deep Learning
2 %pip install tensorflow
3
4 #ติดตั้ง Keras ทำให้สามารถสร้างโมเดล Deep Learning ได้ง่ายขึ้น
5 %pip install keras
6
7 #ติดตั้ง OpenCV สำหรับการประมวลผลภาพและการทำงานกับวิดีโอ
8 %pip install opencv-python==4.13.0.92
```

Python

○ ส่วนของการสร้างโมเดล CNN

```
1  # Import python library
2  from tensorflow.keras.utils import load_img, img_to_array
3  from tensorflow.keras.models import Sequential
4  from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
5  from tensorflow.keras.optimizers import Adam
6  from tensorflow.keras.utils import to_categorical
7  from sklearn.model_selection import StratifiedKFold
8  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
9  import os
10 import numpy as np
11
12 # Define variables
13 path = "data/celeb"
14 model_path = "model/cnn_celeb.keras" # Path to save the final model
15 label_index_path = "model/cnn_label_index.txt" # Path to save mapping between class names and indices
16 img_size = (224, 224)
17
18 # Define Loading data function
19 def load_data():
20     images, labels = [], []
21     label_index = {name: idx for idx, name in enumerate(os.listdir(path))}
22     print(f"Label-index Mapping: {label_index}")
23
24     for label in os.listdir(path):
25         folder_path = os.path.join(path, label)
26         for file in os.listdir(folder_path):
27             img_path = os.path.join(folder_path, file)
28             img = load_img(img_path, target_size=img_size)
29             img = img_to_array(img) / 255.0 # Normalize
30             images.append(img) # Create images dataset
31             labels.append(label_index[label]) # Create labels based on folder names
32
33     return np.array(images), np.array(labels), label_index
34
```

```

35 # Define creating model function
36 def create_cnn_model(input_shape, num_classes):
37     model = Sequential([
38         Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=input_shape),
39         MaxPooling2D((2, 2)),
40
41         Conv2D(64, (3, 3), padding='same', activation='relu'),
42         MaxPooling2D((2, 2)),
43
44         Conv2D(128, (3, 3), padding='same', activation='relu'),
45         MaxPooling2D((2, 2)),
46
47         Flatten(),
48         Dense(256, activation='relu'),
49         Dropout(0.5),
50         Dense(num_classes, activation='softmax')
51     ])
52
53     model.compile(optimizer=Adam(learning_rate=0.0001),
54                 loss='categorical_crossentropy',
55                 metrics=['accuracy'])
56     return model
57
58
59 # Create a datasets
60 X, y, label_index = load_data()
61 y = to_categorical(y) # Convert labels to one-hot encoding
62
63 # Perform 3-fold cross-validation
64 ss = StratifiedKFold(n_splits=3, shuffle=True, random_state=42).split(X, np.argmax(y, axis=1))
65 input_shape = (224, 224, 3)
66 num_classes = y.shape[1]
67

```

```

68 results = []
69 for fold, (train_idx, val_idx) in enumerate(ss):
70     print(f"Training Fold {fold+1}...")
71     X_train, X_val = X[train_idx], X[val_idx]
72     y_train, y_val = y[train_idx], y[val_idx]
73
74     # Create and train model
75     model = create_cnn_model(input_shape, num_classes)
76     model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=10, batch_size=32, verbose=1)
77
78     # Evaluate model
79     y_pred = model.predict(X_val)
80     y_pred_classes = np.argmax(y_pred, axis=1)
81     y_true_classes = np.argmax(y_val, axis=1)
82
83     accuracy = accuracy_score(y_true_classes, y_pred_classes)
84     precision = precision_score(y_true_classes, y_pred_classes, average='weighted')
85     recall = recall_score(y_true_classes, y_pred_classes, average='weighted')
86     f1 = f1_score(y_true_classes, y_pred_classes, average='weighted')
87
88     results.append({
89         "fold": fold+1,
90         "accuracy": accuracy,
91         "precision": precision,
92         "recall": recall,
93         "f1_score": f1
94     })
95
96

```

```

97 # Display the evaluation results in each fold
98 print("\n=== Cross-Validation Results ===")
99 for result in results:
100     print(f"Fold {result['fold']}: ")
101         f"Accuracy={result['accuracy']:.4f}, "
102         f"Precision={result['precision']:.4f}, "
103         f"Recall={result['recall']:.4f}, "
104         f"F1 Score={result['f1_score']:.4f}")
105
106 # Display the average results across all folds
107 avg_accuracy = np.mean([r['accuracy'] for r in results])
108 avg_precision = np.mean([r['precision'] for r in results])
109 avg_recall = np.mean([r['recall'] for r in results])
110 avg_f1_score = np.mean([r['f1_score'] for r in results])
111
112 print(f"\nAverage Accuracy: {avg_accuracy:.4f}")
113 print(f"Average Precision: {avg_precision:.4f}")
114 print(f"Average Recall: {avg_recall:.4f}")
115 print(f"Average F1 Score: {avg_f1_score:.4f}")
116
117 # Retrain the final model on the entire dataset
118 print("\nRetraining final model on the entire dataset...")
119 final_model = create_cnn_model(input_shape, num_classes)
120 final_model.fit(X, y, epochs=10, batch_size=32, verbose=1)
121
122 # Save the model in .keras format
123 final_model.save(model_path)
124 print(f"\nFinal model saved to {model_path}")
125
126 # Save the labels to a .txt file
127 with open(label_index_path, 'w') as f:
128     for label, idx in label_index.items():
129         f.write(f"{label}:{idx}\n")
130 print(f"Labels saved to {label_index_path}")

```

ดูโครงสร้างของ CNN Model

```
1 model.summary()
```

✓ 0.0s

Python

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d_12 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_13 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_14 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten_4 (Flatten)	(None, 100352)	0
dense_8 (Dense)	(None, 256)	25,690,368
dropout_4 (Dropout)	(None, 256)	0
dense_9 (Dense)	(None, 10)	2,570

Total params: 77,358,560 (295.10 MB)

Trainable params: 25,786,186 (98.37 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 51,572,374 (196.73 MB)

○ ส่วนของการทดสอบโมเดล CNN ด้วยภาพ

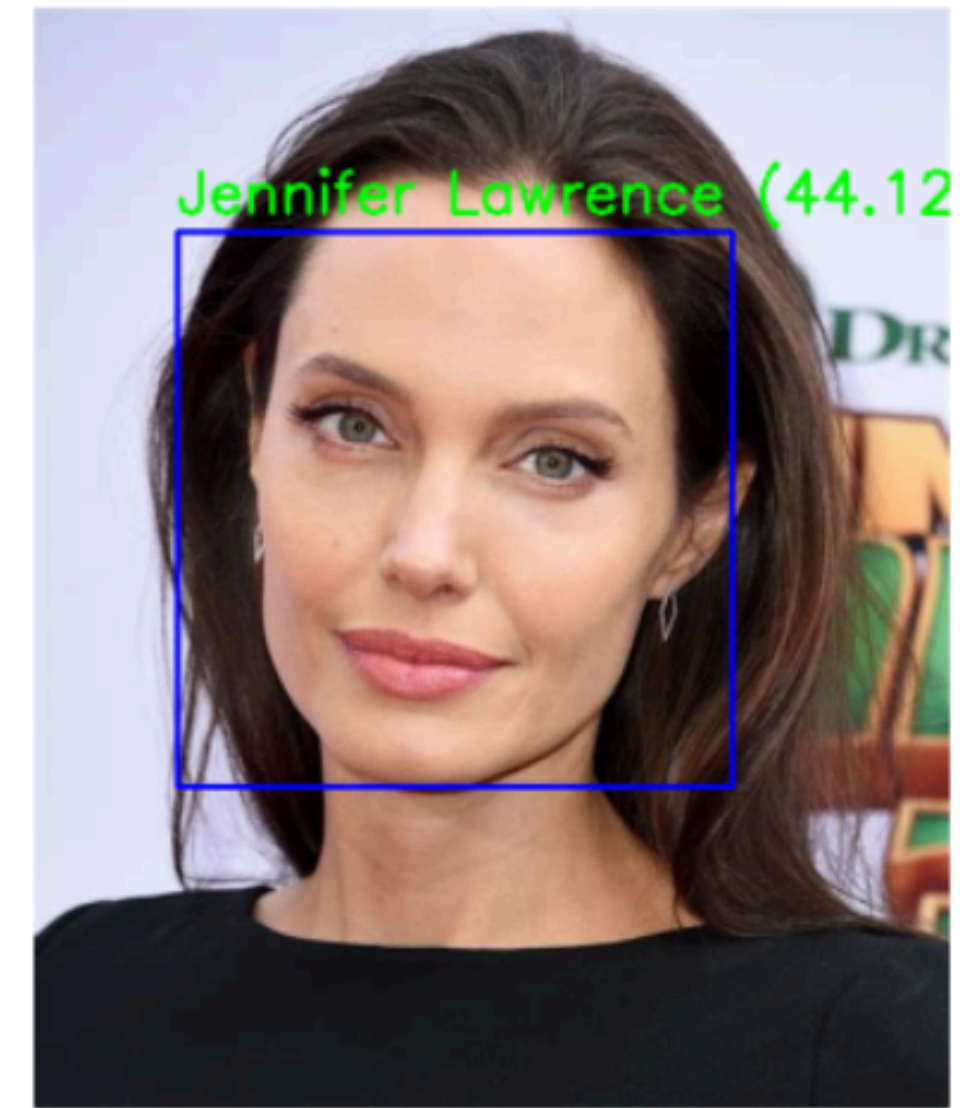
```
1 from tensorflow.keras.models import load_model # type: ignore
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Define variables
7 model_path = "model/cnn_celeb.keras" # Path of the final model
8 label_index_path = "model/cnn_label_index.txt" # Path of mapping between class names and indices
9 image_path = r"data/celeb/Angelina Jolie/001_fe3347c0.jpg" # Path to the input image
10
11 # Load the trained model
12 model = load_model(model_path)
13
14 # Load the label-index mapping from the .txt file
15 label_index = {}
16 index_label = {}
17 with open(label_index_path, 'r') as f:
18     for line in f:
19         label, idx = line.strip().split(":")
20         idx = int(idx)
21         label_index[label] = idx
22         index_label[idx] = label # Reverse the mapping
23
24 #print(f"Loaded label-index: {label_index}")
25 #print(f"Loaded index-label: {index_label}")
26
27 # Load Haar Cascade for face detection
28 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")
29
30 # Load and process the image
31 image = cv2.imread(image_path)
32
33 if image is None:
34     print(f"Error: Could not load image from {image_path}.")
35     exit()
36
37 # Convert the image to grayscale for face detection
38 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
39
```

```

40 # Detect faces in the image
41 faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minSize=(30, 30))
42
43 if len(faces) == 0:
44     print("No face detected.")
45 else:
46     for (x, y, w, h) in faces:
47         # Draw a rectangle around the face
48         cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)
49
50         # Crop the face from the image
51         face_region = image[y:y+h, x:x+w]
52
53         # Convert the face region to RGB and resize it for the model input
54         rgb_face = cv2.cvtColor(face_region, cv2.COLOR_BGR2RGB)
55         resized_face = cv2.resize(rgb_face, (224, 224)) / 255.0 # Normalize
56         input_face = np.expand_dims(resized_face, axis=0) # Add batch dimension
57
58         # Predict using the model
59         predictions = model.predict(input_face)
60         index = np.argmax(predictions) # Get the index of the highest confidence
61         label = index_label.get(index, "Unknown") # Use index_label to get the label
62         confidence = predictions[0][index] # Confidence score
63
64         # Add the label and confidence to the image
65         display_text = f"{label} ({confidence*100:.2f}%)"
66         cv2.putText(image, display_text, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
67
68 # Convert BGR to RGB for displaying with matplotlib
69 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
70
71 # Show the image with detected faces and predictions using matplotlib
72 plt.imshow(image_rgb)
73 plt.axis('off') # Hide axes
74 plt.title("Image - Face Recognition")
75 plt.show()

```

Image - Face Recognition



○ ส่วนของการทดสอบโมเดล CNN ด้วย Webcam

```
1  from tensorflow.keras.models import load_model # type: ignore
2  import cv2
3  import numpy as np
4
5  # Define variables
6  model_path = "model/cnn_celeb.keras" # Path of the final model
7  label_index_path = "model/cnn_label_index.txt" # Path of mapping between class names and indices
8
9  # Load the trained model
10 model = load_model(model_path)
11
12 # Load the label-index mapping from the .txt file
13 label_index = {}
14 index_label = {}
15 with open(label_index_path, 'r') as f:
16     for line in f:
17         label, idx = line.strip().split(":")
18         idx = int(idx)
19         label_index[label] = idx
20         index_label[idx] = label # Reverse the mapping
21
22 #print(f"Loaded label-index: {label_index}")
23 #print(f"Loaded index-label: {index_label}")
24
25 # Load Haar Cascade for face detection
26 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
27
28 # Webcam feed
29 cap = cv2.VideoCapture(0) # Open webcam (0 is the default camera)
30
31 # Check if the webcam opened successfully
32 if not cap.isOpened():
33     print("Error: Could not access the webcam.")
34     exit()
```

```

35
36 print("Press 'q' to quit the webcam.")
37 while True:
38     # Read a frame from the webcam
39     ret, frame = cap.read()
40     if not ret:
41         print("Failed to grab a frame.")
42         break
43
44     # Convert the frame to grayscale for face detection
45     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46
47     # Detect faces in the frame
48     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minSize=(30, 30))
49
50     if len(faces) == 0:
51         print("No face detected.")
52     else:
53         for (x, y, w, h) in faces:
54             # Draw a rectangle around the face
55             cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
56
57             # Crop the face from the frame
58             face_region = frame[y:y+h, x:x+w]
59
60             # Convert the face region to RGB and resize it for the model input
61             rgb_face = cv2.cvtColor(face_region, cv2.COLOR_BGR2RGB)
62             resized_face = cv2.resize(rgb_face, (224, 224)) / 255.0 # Normalize
63             input_face = np.expand_dims(resized_face, axis=0) # Add batch dimension
64
65             # Predict using the model
66             predictions = model.predict(input_face)
67             index = np.argmax(predictions)
68             label = index_label.get(index, "Unknown") # Use idx_to_label to get the label
69             confidence = predictions[0][index]
70
71             # Add the label and confidence to the frame
72             display_text = f"{label} ({confidence*100:.2f}%)"
73             cv2.putText(frame, display_text, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

```

```
74
75     # Show the frame in a window
76     cv2.imshow("Webcam - Face Recognition", frame)
77
78     # Exit the webcam feed when 'q' is pressed
79     if cv2.waitKey(1) == ord('q'):
80         break
81
82 # Release the webcam and close windows
83 cap.release()
84 cv2.destroyAllWindows()
```

4.4.4 Pre-trained CNN Model

Pre-trained model คือ โมเดลที่ผ่านการ Train บน Datasets ขนาดใหญ่มาแล้ว และสามารถนำมา Fine-tune หรือปรับใช้กับงานใหม่ได้ โดยไม่ต้องเริ่ม Train ใหม่จากศูนย์

Pre-trained CNN model คือ Pre-trained model ที่มีโครงสร้างแบบ CNN

ตารางที่ 4-3 : ตัวอย่างของ Pre-trained CNN model

Model	Main Version	Description
VGGNet	11, 13, 16, 19	โครงสร้างเข้าใจง่ายคล้ายกับ Classic CNN
ResNet	18, 34, 50, 101, 152	แก้ปัญหา Vanishing Gradient
Inception	V1-V4, Inception-ResNet	มีความหลากหลายในการทำ Feature Extraction
DenseNet	121, 161, 169, 201	นำ Feature Maps จากทุกชั้นก่อนหน้า มาใช้ซ้ำในชั้นถัดไป
MobileNet	V1-V4	ออกแบบมาสำหรับอุปกรณ์ที่มีทรัพยากรจำกัด (เช่น มือถือ)
EfficientNet	B0-B7, V2	เน้นการค้นหาสมดุลระหว่างประสิทธิภาพและความแม่นยำ

4.5 รีเคอร์เนต นิวรัลเน็ตเวิร์ก (RNN)

รีเคอร์เนต นิวรัลเน็ตเวิร์ก (Recurrent Neural Network: RNN)

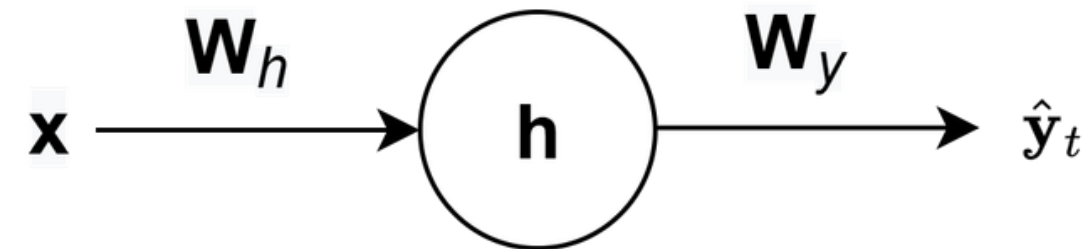
- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Recurrent neural network
รับข้อมูลเข้ามาแบบลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือสัญญาณเวลา โดยส่งข้อมูลผ่านชั้นต่าง ๆ เช่น Recurrent Layer (เช่น Simple RNN, LSTM, GRU) และ Fully Connected Layer
- เรียนรู้โดยใช้ Backpropagation Through Time (BPTT)
เป็นการปรับปรุงความรู้หรือน้ำหนัก (Weight) ย้อนกลับตามลำดับเวลา โดยใช้ค่า error จาก output แต่ละช่วงเวลามาคำนวณย้อนกลับ
- มีคุณสมบัติในการจดจำลำดับและบริบท (Sequence and Context Memory)
สามารถเก็บข้อมูลจากขั้นตอนก่อนหน้า (Previous Steps) เพื่อใช้ในการประมวลผลขั้นตอนถัดไป ทำให้เข้าใจความสัมพันธ์ในลำดับข้อมูลได้
- นิยมใช้ในงานประมวลผลข้อมูลลำดับ (Sequential Data Processing)
เช่น Time Series Forecasting, Natural Language Processing (NLP), Machine Translation

4.5.1 โครงสร้างของ RNN

- **Feedforward Network (นูนวน)**

$$\mathbf{h} = \varphi_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}} = \varphi_y(\mathbf{W}_y \mathbf{h} + \mathbf{b}_y)$$

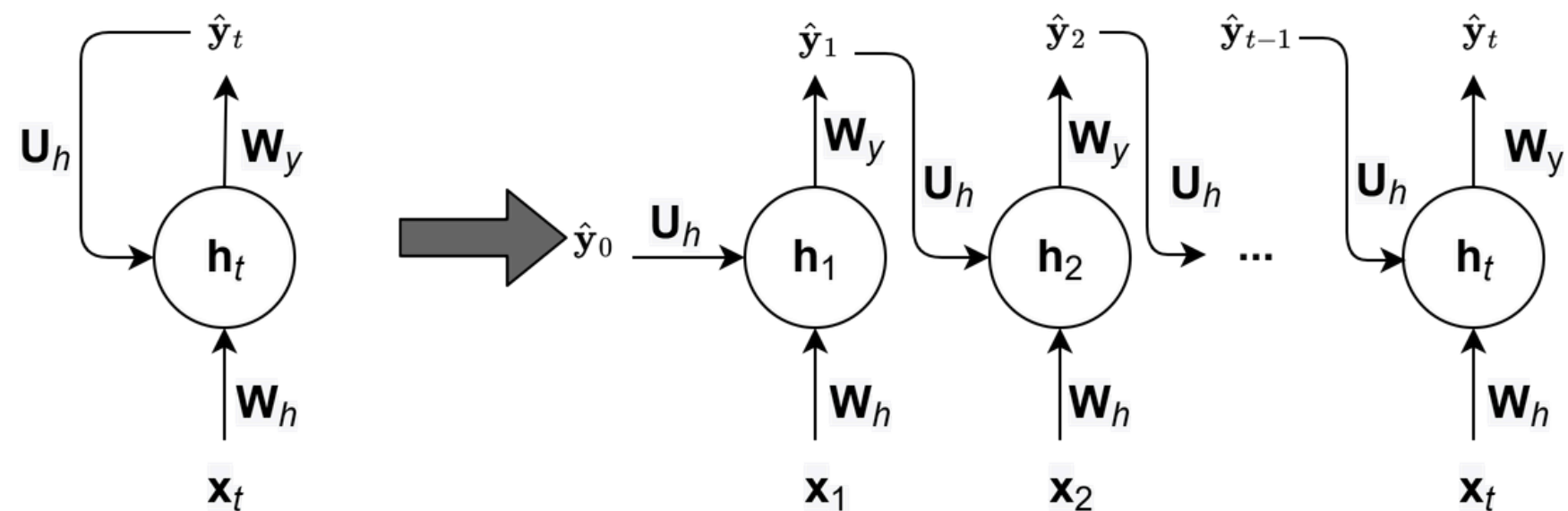


ภาพที่ 4-6 : โครงสร้างของ Feedforward Network

- **Jordan Recurrent Network**

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{y}_{t-1} + \mathbf{b}_h)$$

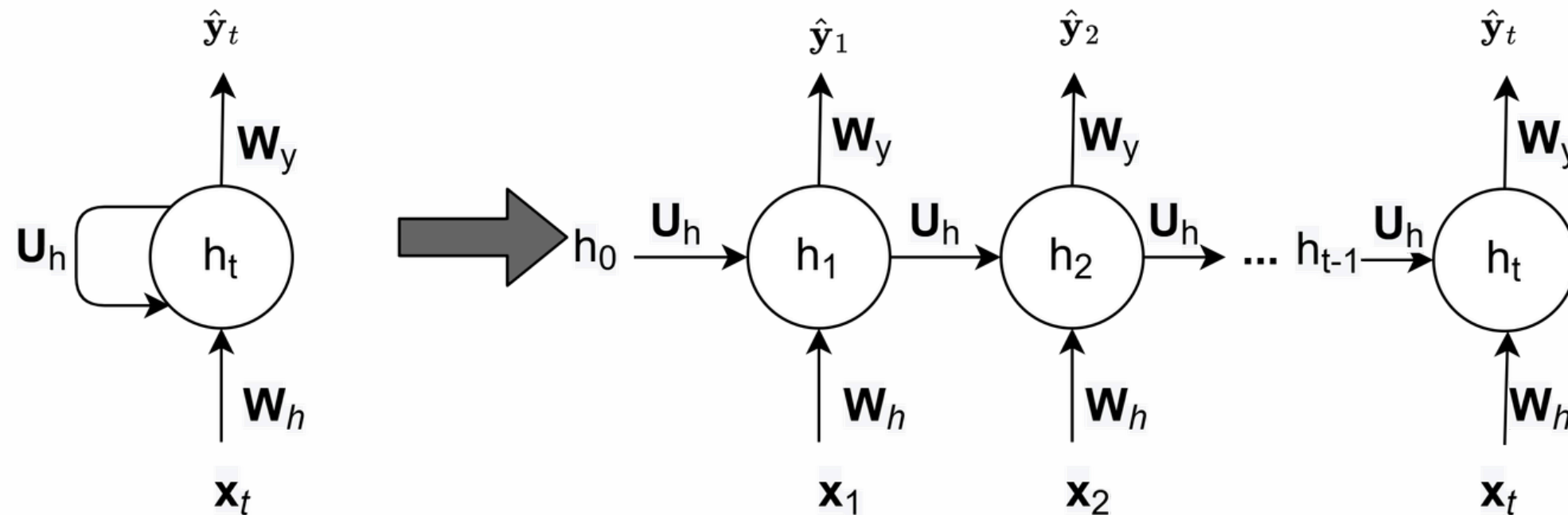
$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ภาพที่ 4-7 : โครงสร้างของ Jordan Recurrent Network

- **Elman Recurrent Network**

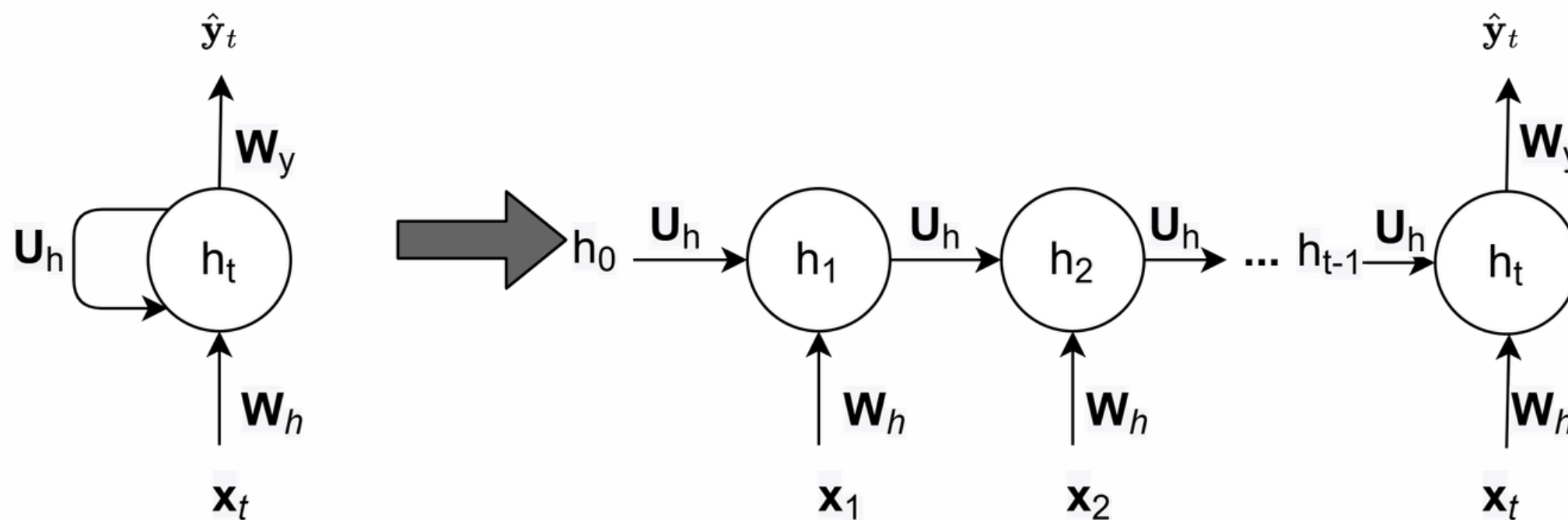
- แบบ Many to Many



$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

- แบบ Many to One



ภาพที่ 4-8 : โครงสร้างของ Elman Recurrent Network

4.5.2 ตัวอย่างการคำนวณ RNN (Elman Network)

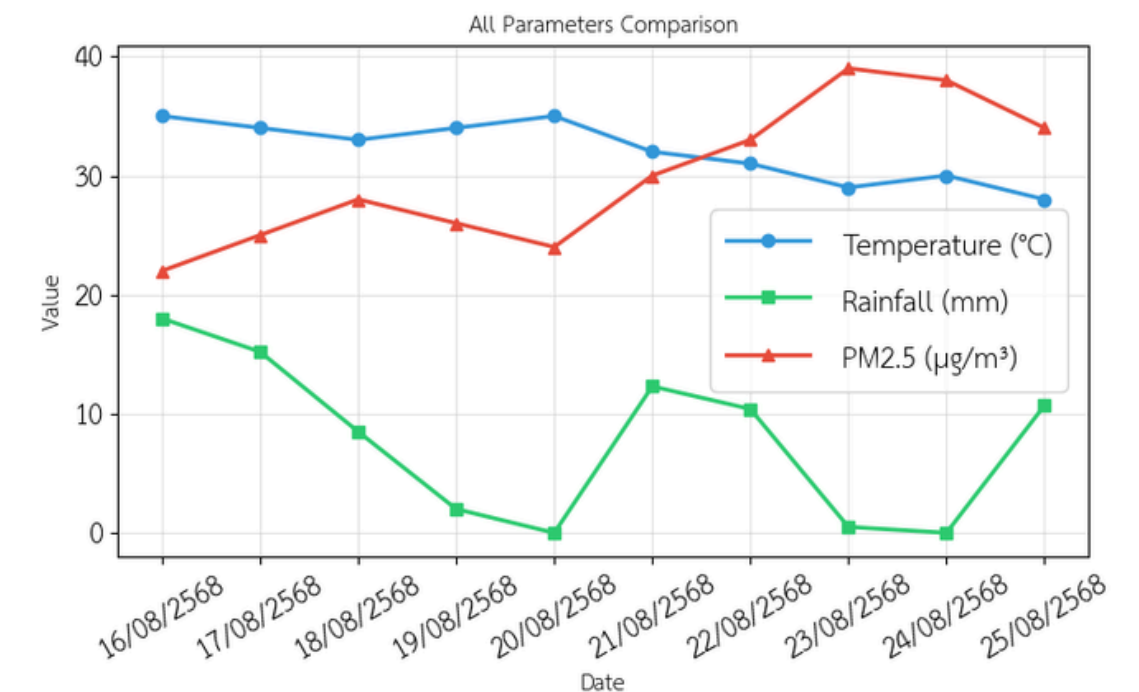
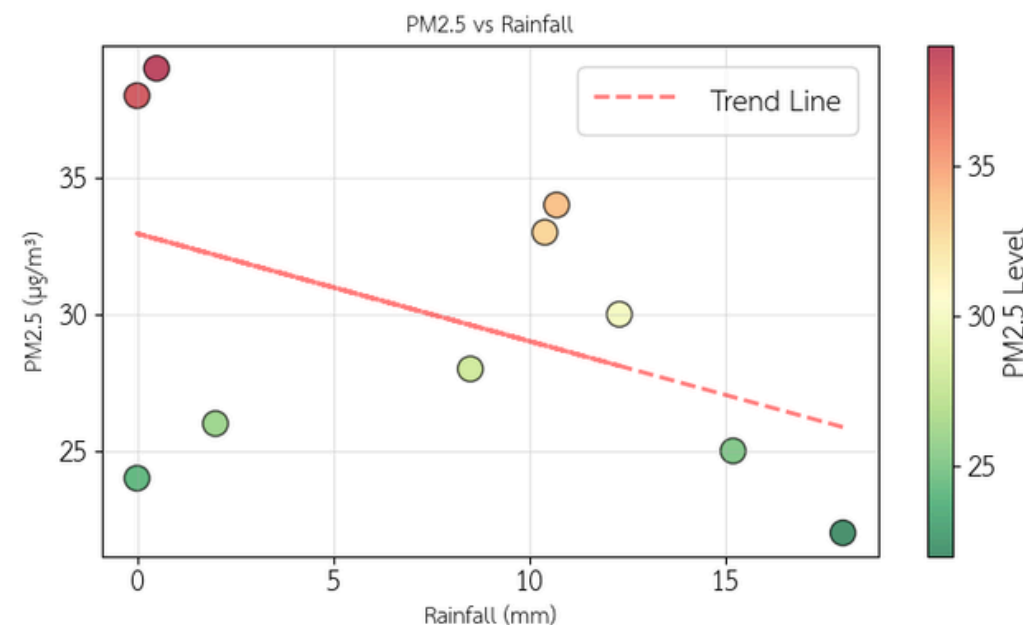
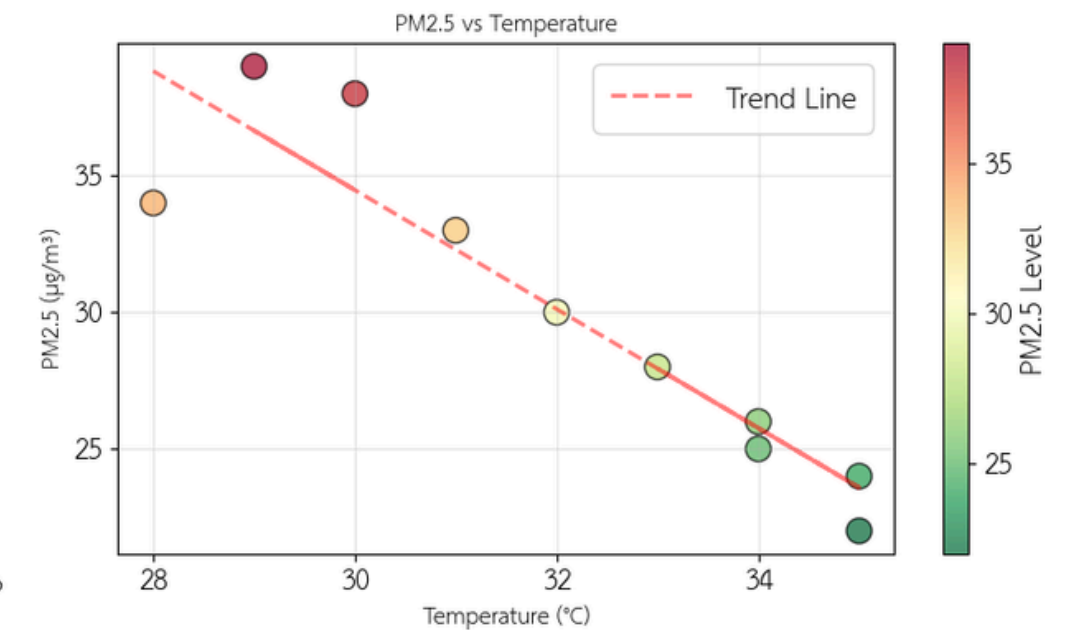
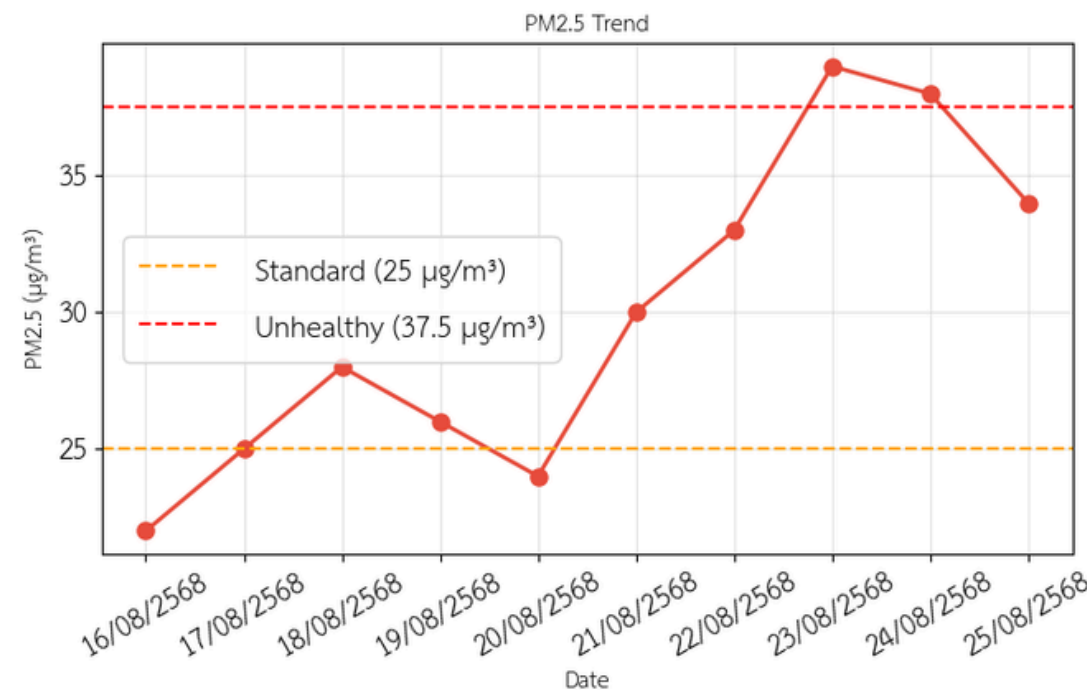
ข้อที่ 1 : แสดงวิธีการคำนวณ RNN (Elman Network) เพื่อทำนายปริมาณ PM2.5 ณ วันที่ 19/8/2568

กำหนดให้

- Input Node = 3,
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (ReLU)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [-0.05, 0.08, 0.1]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.2$
- Hidden Bias: $\mathbf{b}_h = 0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 20$
- Output Bias: $\mathbf{b}_y = 7$

ตารางที่ 4-4 : ชุดข้อมูล PM2.5

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 ($\mu\text{g}/\text{m}^3$)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28
19/8/2568	34	2	26
20/8/2568	35	0	24
21/8/2568	32	12.3	30
22/8/2568	31	10.4	33
23/8/2568	29	0.5	39
24/8/2568	30	0	38
25/8/2568	28	10.7	34



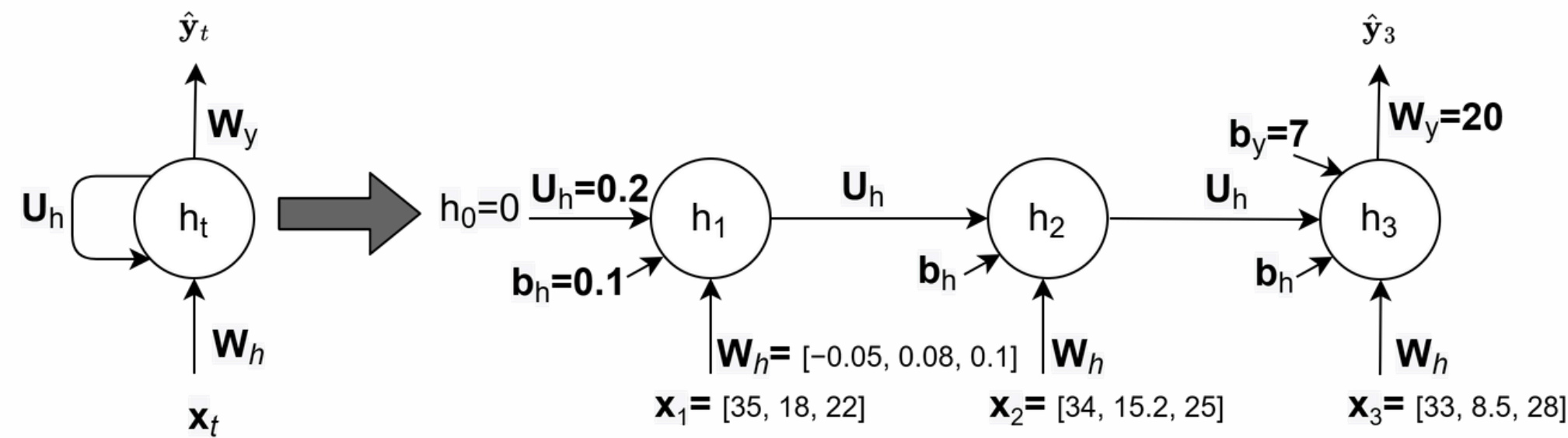
ภาพที่ 4-9 : ชุดข้อมูล PM2.5

วิธีทำ

กำหนดให้

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ค่า $\mathbf{W}_h$, $\mathbf{U}_h$, และ $\mathbf{b}_h$ จะเท่าเดิม

	วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)
$\mathbf{x}_1$	16/8/2568	35	18	22
$\mathbf{x}_2$	17/8/2568	34	15.2	25
$\mathbf{x}_3$	18/8/2568	33	8.5	28

- คำนวณใน Hidden Layer (State)

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

เมื่อ $t=1$ (16/08/2568)

$$Z_1 = -0.05 \times 35 + 0.08 \times 18 + 0.1 \times 22 + 0.2 \times 0 + 0.1 = 1.99$$

$$\mathbf{h}_1 = \frac{1}{1 + e^{-1.99}} = 0.88$$

เมื่อ $t=2$ (17/08/2568)

$$Z_2 = -0.05 \times 34 + 0.08 \times 15.2 + 0.1 \times 25 + 0.2 \times 0.88 + 0.1 = 2.292$$

$$\mathbf{h}_2 = \frac{1}{1 + e^{-2.292}} = 0.908$$

เมื่อ $t=3$ (18/08/2568)

$$Z_3 = -0.05 \times 33 + 0.08 \times 8.5 + 0.1 \times 28 + 0.2 \times 0.908 + 0.1 = 2.112$$

$$\mathbf{h}_3 = \frac{1}{1 + e^{-2.112}} = 0.892$$

เมื่อ $\mathbf{W}_h = [-0.05, 0.08, 0.1]$

$\mathbf{U}_h = 0.2$ $\mathbf{h}_0 = 0$ $\mathbf{b}_h = 0.1$

$\mathbf{x}_1 = [35, 18, 22]$

$\mathbf{x}_2 = [34, 15.2, 25]$

$\mathbf{x}_3 = [33, 8.5, 28]$

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28

- **คำนวณใน Output Layer**

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

$$\hat{\mathbf{y}}_3 = \text{ผลการทำนายของวันที่ 19/08/2568}$$

$$\hat{\mathbf{y}}_3 = \varphi_y(20 \times 0.892 + 7) = \varphi_y(24.84)$$

$$\hat{\mathbf{y}}_3 = \max(0, 24.84) = 24.84$$

เมื่อ $\mathbf{W}_y = 20$
 $\mathbf{h}_3 = 0.892$
 $\mathbf{b}_y = 7$

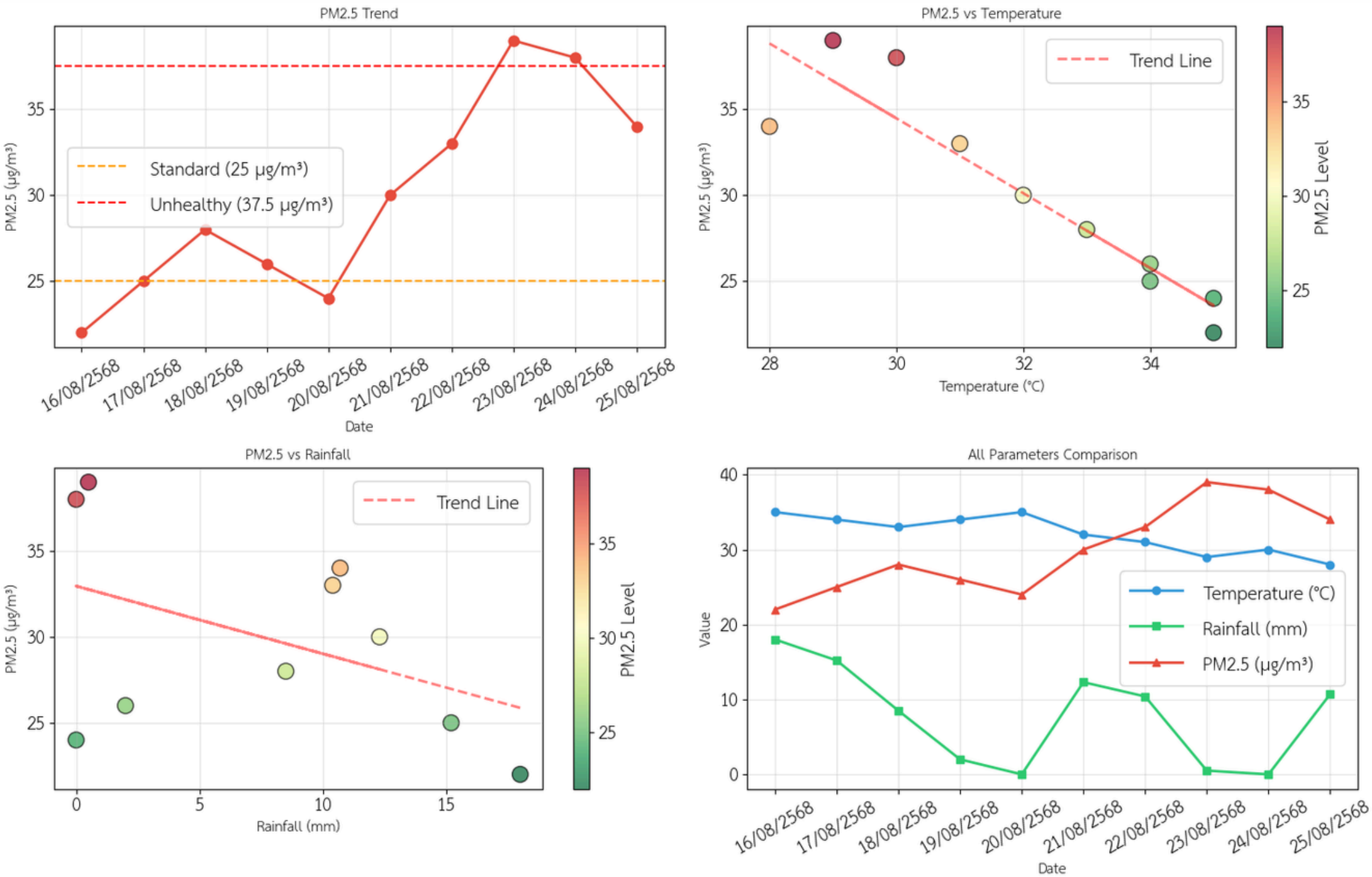
ตอบ ผลการทำนายปริมาณ PM2.5 ณ วันที่ 19/8/2568 คือ $24.84 \mu\text{g}/\text{m}^3$

ข้อที่ 2 : จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายแนวโน้มของค่า PM2.5 (สูงขึ้น/ต่ำลง) ณ วันที่ 19/8/2568
กำหนดให้

- Input Node = 3,
- Hidden Node = 1 (ReLU)
- Output Node = 1 (Logistic Sigmoid)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $W_h=[0.2, -0.1, 0.3]$
- Hidden Recurrent Weight: $U_h=0.4$
- Hidden Bias: $b_h=0.1$
- Output Weight (Hidden $\rightarrow$ Output): $W_y=0.8$
- Output Bias: $b_y=-0.3$

ตารางที่ 4-5 : ชุดข้อมูล PM2.5

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)	change
16/8/2568	35	18	22	ขึ้น
17/8/2568	34	15.2	25	ขึ้น
18/8/2568	33	8.5	28	ขึ้น
19/8/2568	34	2	26	ลง
20/8/2568	35	0	24	ลง
21/8/2568	32	12.3	30	ขึ้น
22/8/2568	31	10.4	33	ขึ้น
23/8/2568	29	0.5	39	ขึ้น
24/8/2568	30	0	38	ลง
25/8/2568	28	10.7	34	ลง



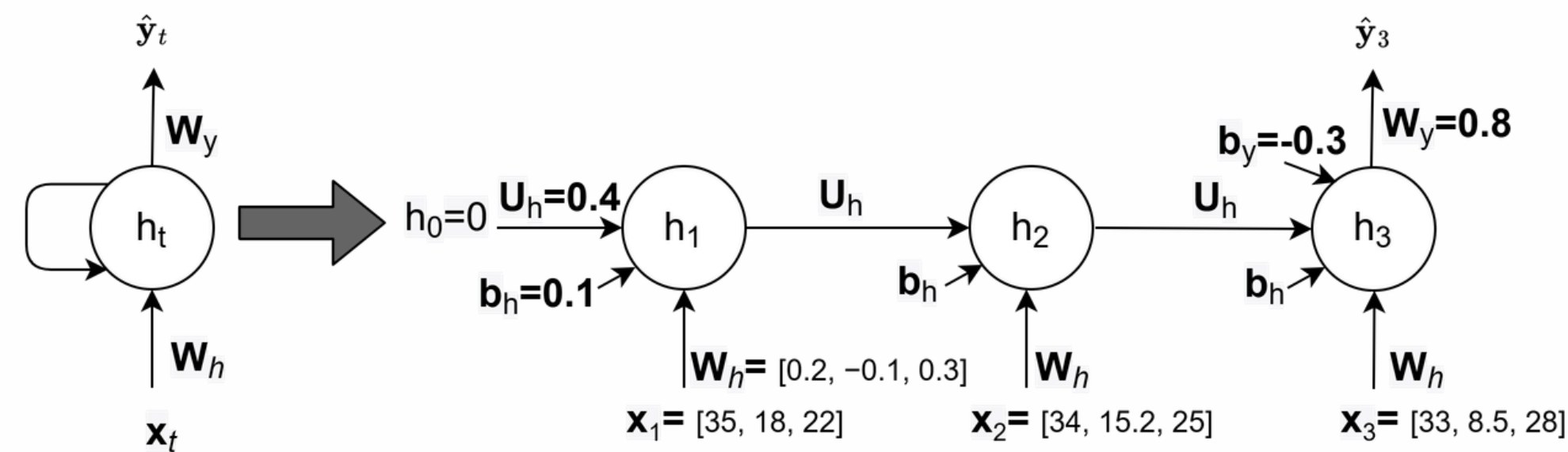
ภาพที่ 4-10 : ชุดข้อมูล PM2.5

วิธีทำ

กำหนดให้

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ค่า $\mathbf{W}_h$, $\mathbf{U}_h$, และ $\mathbf{b}_h$ จะเท่าเดิม

	วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)	change
X₁	16/8/2568	35	18	22	ขึ้น
X₂	17/8/2568	34	15.2	25	ขึ้น
X₃	18/8/2568	33	8.5	28	ขึ้น

- คำนวณใน Hidden Layer (State)

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

เมื่อ $t=1$ (16/08/2568)

$$Z_1 = 0.2 \times 35 + (-0.1) \times 18 + 0.3 \times 22 + 0.4 \times 0 + 0.1 = 11.9$$

$$\mathbf{h}_1 = \max(11.9, 0) = 11.9$$

เมื่อ $t=2$ (17/08/2568)

$$Z_2 = 0.2 \times 34 + (-0.1) \times 15.2 + 0.3 \times 25 + 0.4 \times 11.9 + 0.1 = 17.64$$

$$\mathbf{h}_2 = \max(17.64, 0) = 17.64$$

เมื่อ $t=3$ (18/08/2568)

$$Z_3 = 0.2 \times 33 + (-0.1) \times 8.5 + 0.3 \times 28 + 0.4 \times 17.64 + 0.1 = 21.306$$

$$\mathbf{h}_3 = \max(21.306, 0) = 21.306$$

เมื่อ $\mathbf{W}_h = [0.2, -0.1, 0.3]$

$\mathbf{U}_h = 0.4$ $\mathbf{h}_0 = 0$ $\mathbf{b}_h = 0.1$

$\mathbf{x}_1 = [35, 18, 22]$

$\mathbf{x}_2 = [34, 15.2, 25]$

$\mathbf{x}_3 = [33, 8.5, 28]$

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28

- **คำนวณใน Output Layer**

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

$$\hat{\mathbf{y}}_3 = \text{ผลการทำนายของวันที่ 19/08/2568}$$

$$\hat{\mathbf{y}}_3 = \varphi_y(0.8 \times 21.306 - 0.3) = \varphi_y(16.7448)$$

$$\hat{\mathbf{y}}_3 = \frac{1}{1 + e^{-16.7448}} = 0.999999995$$

ดังนั้น ในวันที่ 19/8/2568 มีแนวโน้มว่าค่า PM2.5 จะสูงขึ้น

เมื่อ $\mathbf{W}_y = 0.8$
 $\mathbf{h}_3 = 21.306$
 $\mathbf{b}_y = -0.3$

4.6 ลองชอร์ตเทอมเมมโมรี (LSTM)

ลองชอร์ตเทอมเมมโมรี (Long Short-Term Memory Network: LSTM)

- **เป็น Recurrent Neural Network ชนิดพิเศษ**
ออกแบบมาเพื่อแก้ปัญหาเรื่อง การจำระยะยาว (Long-Term Memory) ของ RNN โดยสามารถรับข้อมูลแบบลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือสัญญาณเวลา แล้วส่งข้อมูลผ่านโครงสร้างหลักที่มี Cell State และ Gate ต่าง ๆ (Input Gate, Forget Gate, Output Gate) เพื่อควบคุมการเก็บและลืมข้อมูล ก่อนจะส่งต่อไปยัง Fully Connected Layer เพื่อคำนวณผลลัพธ์สุดท้าย
- **เรียนรู้โดยใช้ Backpropagation Through Time (BPTT)**
เป็นการปรับปรุงความรู้หรือน้ำหนัก (Weight) ย้อนกลับตามลำดับเวลา (Time Step) โดยใช้ค่า error ที่คำนวณจาก Actual/Predicted Outputs ในแต่ละช่วงเวลามาคำนวณย้อนกลับ เช่นเดียวกับกับ RNN แต่มีการจัดการปัญหา Gradient Vanishing โดยใช้โครงสร้างแบบ “Cell” ประกอบด้วย Gate 3 ประเภท ดังนี้
 - Forget Gate: ควบคุมข้อมูลที่ควรถูกลืมจากสถานะก่อนหน้า
 - Input Gate: ควบคุมข้อมูลใหม่ที่จะถูกเพิ่มเข้าไปในหน่วยความจำ
 - Output Gate: ควบคุมข้อมูลที่จะถูกส่งออกไปยังชั้นถัดไป
- **นิยมใช้ในงานประมวลผลข้อมูลลำดับ (Sequential Data Processing)**
เช่น Time Series Forecasting, Natural Language Processing (NLP), Machine Translation

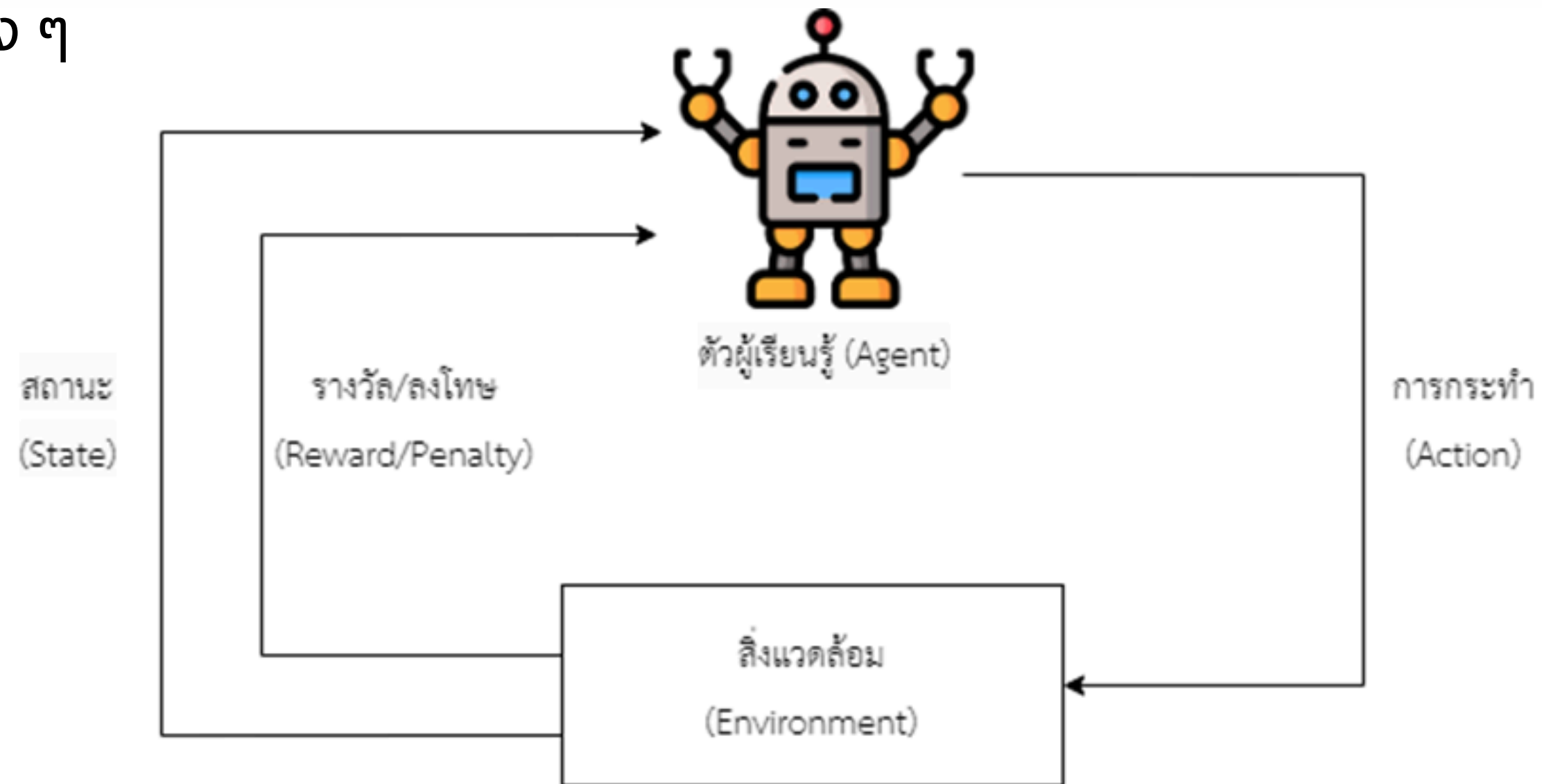
ปัญหา Vanishing Gradient เป็นปัญหาที่เกิดขึ้นในระบบการ Backpropagation โดยเฉพาะใน RNN ที่มีจำนวน Time Step ยาว เมื่อมีการคำนวณย้อนกลับตามลำดับของข้อมูล (Backpropagation Through Time) ค่ากราดีเอนต์ (Gradient) จะถูก “คูณซ้ำ ๆ” ด้วยค่าที่มีขนาดเล็กกว่า 1 เช่น $0.9 \times 0.8 \times 0.5 \times \dots$ ส่งผลให้ค่า Gradient ค่อย ๆ เล็กลงจนเกือบเป็นศูนย์ ทำให้เครือข่าย (Network) ไม่สามารถปรับน้ำหนัก (Weight) ได้อย่างมีประสิทธิภาพ และไม่สามารถเรียนรู้จากข้อมูลในช่วงเวลายาว ๆ ได้อย่างเหมาะสม

4.7 ดีป คว-เน็ตเวิร์ก (DQN)

ดีป คว-เน็ตเวิร์ก (Deep Q-Network: DQN)

- **เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)**
เป็นการผสมการทำงานระหว่าง ANN กับแนวคิดของ Reinforcement Learning แบบ Q-Learning เพื่อให้ตัวแทน (Agent) สามารถเรียนรู้การตัดสินใจที่เหมาะสมที่สุดจากการโต้ตอบกับสภาพแวดล้อม (Environment) โดยปรับปรุงนโยบายการตัดสินใจ (Policy) ผ่านค่ารางวัล (Reward) หรือการลงโทษ (Penalty) ที่ได้รับจากแต่ละการกระทำ (Action)
- **เรียนรู้โดยใช้ Q-Learning + Backpropagation**
ย้อนกลับไปปรับ weight ของโครงข่ายประสาทเทียม โดยใช้ค่า Error ที่ได้จากส่วนต่างระหว่างค่าเป้าหมาย (Target Q-value) ที่คำนวณได้จาก Q-Learning กับค่าทำนาย (Predicted Q-value) ที่ได้จาก ANN
- **นิยมใช้ในระบบอัตโนมัติที่มีการปฏิสัมพันธ์กับสภาพแวดล้อม**
เช่น เกมกระดาน (หมากออส, หมากรูก), การควบคุมหุ่นยนต์ (Robot Control), การจราจรอัจฉริยะ (Smart Traffic Control), ระบบแนะนำอัตโนมัติ (Recommendation Systems)

การเรียนรู้แบบ Reinforcement เป็นเทคนิคที่เอเจนต์ (Agent) หรือตัวแทน ได้รับการกระตุ้น หรือเรียนรู้จากสภาพแวดล้อม (Environment) ที่อยู่รอบตัว เมื่อเอเจนต์ทำในสิ่งที่ถูกต้องหรือเหมาะสมก็จะได้รับรางวัล (Reward) แต่เมื่อทำในสิ่งที่ผิดก็จะถูกลงโทษ (Penalty) กระบวนการนี้จะช่วยให้เอเจนต์ปรับปรุงพฤติกรรมของตนเองอย่างต่อเนื่อง เพื่อให้สามารถตัดสินใจได้ดีที่สุดในสถานการณ์ต่าง ๆ



ภาพที่ 4-11 : กระบวนการการเรียนรู้แบบ Reinforcement

4.8 เจเนอเรทีฟ พรีเทรนด ทรานส์ฟอร์มเมอร์ (GPT)

เจเนอเรทีฟ พรีเทรนด ทรานส์ฟอร์มเมอร์ (Generative Pre-trained Transformer: GPT)

- **เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)**
เป็น ANN ที่มีสถาปัตยกรรมแบบ Transformer ซึ่งใช้กลไก Self-Attention เพื่อให้โมเดลสามารถเข้าใจบริบทของข้อมูลแบบลำดับ (Sequential Data) ได้อย่างมีประสิทธิภาพ โดย GPT ได้รับการฝึกสอนล่วงหน้า (Pre-trained) บนข้อมูลจำนวนมากมหาศาลเพื่อเรียนรู้โครงสร้างของภาษา แล้วจึงนำไปปรับใช้กับงานเฉพาะทาง (Fine-tuning)
- **เรียนรู้โดยใช้ Self-Attention + Backpropagation**
ใช้กลไก Self-Attention เพื่อคำนวณความสัมพันธ์ระหว่างคำในลำดับ และใช้ Backpropagation ปรับ Weight จาก Error ระหว่างค่าจริง (Actual Token) กับค่าทำนาย (Predicted Token) เพื่อให้สามารถสร้างข้อความที่มีความหมายและสอดคล้องกับบริบท
- **นิยมใช้ในงานประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP)**
เช่น Chatbot, Machine Translation, Text Summarization

4.9 เจเนอเรทีฟ แอดเวอร์ซารีชาเรียล เน็ตเวิร์ก (GAN)

เจเนอเรทีฟ แอดเวอร์ซารีชาเรียล เน็ตเวิร์ก (Generative Adversarial Network: GAN)

- **เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)**
เป็นการผสมการทำงานระหว่าง ANN สองชุด ได้แก่ Generator และ Discriminator
 - **Generator (ผู้สร้าง)** : ทำหน้าที่สร้างข้อมูลปลอมที่มีลักษณะคล้ายข้อมูลจริงมากที่สุด
 - **Discriminator (ผู้แยกแยะ)** : ทำหน้าที่ตรวจสอบว่าข้อมูลที่ได้รับเป็นข้อมูลจริงหรือข้อมูลปลอมทั้งสองโครงข่ายเรียนรู้แข่งขันกัน (Adversarial Learning) เพื่อให้ Generator สามารถสร้างข้อมูลที่สมจริงยิ่งขึ้นเรื่อย ๆ
- **เรียนรู้โดยใช้ Adversarial Training + Backpropagation**
ทั้ง Generator และ Discriminator ปรับ Weight ของตนเองผ่านกระบวนการ Backpropagation โดยมีการคำนวณ Error จากผลการทำนายของ Discriminator — Generator จะพยายาม “หลอก” Discriminator ให้เชื่อว่าข้อมูลปลอมเป็นข้อมูลจริง ส่วน Discriminator จะพยายามแยกให้ได้ว่าข้อมูลใดเป็นข้อมูลจริง
- **นิยมใช้ในงานสร้างข้อมูลใหม่ (Data Generation)**
เช่น Face Generation, Image Generation, Audio Generation, Video Generation, Image Enhancement

4.10 แบบฝึกหัด (Exercise)

1. จงแสดงวิธีการคำนวณ MLP เพื่อประเมินราคาบ้าน

กำหนดให้

- Input Node = 3
- Hidden Node = 2 (Logistic Sigmoid)
- Output Node = 1 (Linear)
- Hidden Weights (Input → Hidden): $\mathbf{W}_h = \begin{bmatrix} 3.003 & 2.012 \\ 0.485 & 0.283 \\ 1.944 & 1.955 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [0.573, 1.072]$
- Output Weight (Hidden → Output): $\mathbf{W}_y = \begin{bmatrix} 65.472 \\ 65.005 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [66.383]$

ตารางที่ 4-6ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	7	0.4	3	139
2	4	0.2	7	159
3	3	0.6	4	60
4	10	1.2	2	25
5	8	1.4	1	26
6	3	0.5	5	65
7	2	0.2	3	100
8	3	0.1	7	160
9	2	0.3	2	134
10	9	0.9	1	40

ตารางที่ 4-6ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	5	0.7	4	?

2. จงแสดงวิธีการคำนวณ MLP เพื่อวินิจฉัยโรคเบาหวาน

กำหนดให้

- Input Node = 2
- Hidden Node = 3 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Hidden Weights (Input → Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.062 & 0.259 & 0.13 \\ 0.06 & -0.147 & 0.185 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.079, 0.495, 0.586]$
- Output Weight (Hidden → Output): $\mathbf{W}_y = \begin{bmatrix} -0.089 \\ 0.411 \\ 0.116 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [0.172]$

ตารางที่ 4-7ก : ชุดข้อมูลฝึก (Training Set)

รายการ	ดัชนีมวลกาย (BMI)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	20	90	ไม่เป็นโรค
2	22	100	ไม่เป็นโรค
3	21	88	ไม่เป็นโรค
4	23	85	ไม่เป็นโรค
5	28	135	เป็นโรค
6	30	137	เป็นโรค
7	32	140	เป็นโรค
8	29	123	เป็นโรค
9	31	145	เป็นโรค
10	27	125	เป็นโรค

ตารางที่ 4-7ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ดัชนีมวลกาย (BMI)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	100	?

3. จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายราคาหุ้น ณ วันที่ 15/8/2568

กำหนดให้

- Input Node = 4
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (ReLU)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [0.01, -0.01, 0.02, -0.02]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.5$
- Hidden Bias: $\mathbf{b}_h = -0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 100$
- Output Bias: $\mathbf{b}_y = 10$

ตารางที่ 4-8 : ชุดข้อมูลราคาหุ้น

date	open	high	low	close
1/8/2568	100	102	98	99
4/8/2568	99	100	95	96
5/8/2568	96	98	95	97
6/8/2568	97	97	92	93
7/8/2568	93	95	91	92
8/8/2568	92	94	90	94
11/8/2568	94	94	88	89
12/8/2568	89	90	86	87
13/8/2568	87	88	85	88
14/8/2568	88	88	82	84

4. จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายแนวโน้มราคาหุ้น (สูงขึ้น/ต่ำลง) ณ วันที่ 15/8/2568

กำหนดให้

- Input Node = 4,
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (Logistic Sigmoid)
- Time Step = 2
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [0.01, -0.01, 0.02, -0.02]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.5$
- Hidden Bias: $\mathbf{b}_h = -0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 2$
- Output Bias: $\mathbf{b}_y = -0.5$

ตารางที่ 4-9 : ชุดข้อมูลราคาหุ้น

date	open	high	low	close	change
1/8/2568	100	102	98	99	ลง
4/8/2568	99	100	95	96	ลง
5/8/2568	96	98	95	97	ขึ้น
6/8/2568	97	97	92	93	ลง
7/8/2568	93	95	91	92	ลง
8/8/2568	92	94	90	94	ขึ้น
11/8/2568	94	94	88	89	ลง
12/8/2568	89	90	86	87	ลง
13/8/2568	87	88	85	88	ขึ้น
14/8/2568	88	88	82	84	ลง



THANK → YOU!

Get In Touch With Us!

Phone

0-2692-2360-4

Facebook

<https://www.facebook.com/ComSci.CPC/>

Address

58 ถนนวิภาวดีรังสิต แขวง
รัชดาภิเษก เขตดินแดง

